THE COMPLETE ENCYCLOPEDIA OF

Artificial Intelligence and Robotics Engineering



VOLUME I

Foundations of Artificial Intelligence and Robotics

Introductory manuscript

PRELIMINARY MANUSCRIPT · NOT FOR PUBLICATION

Contents

Preface 3

How to use this volume 4

1. Introduction to Artificial Intelligence 5

2. Robotics Fundamentals 74

3. Machine Learning 91

4. Computer Vision 95

5. Natural Language Processing 98

6. Control Systems 101

Selected reading 104

Glossary 106

Preface

Artificial intelligence and robotics meet wherever a machine must make a decision that matters in the physical world. A model may classify an image correctly, yet a robot using that model can still fail because its camera is blurred, its clock is late, or its brakes are undersized. Engineering therefore requires more than an impressive algorithm: it requires a chain of evidence from a problem statement to a system that behaves acceptably in its intended environment.

This volume introduces six foundations of that chain. The chapters begin with artificial intelligence and the physical structure of robots, then develop machine learning, computer vision, natural language processing, and feedback control. Mathematics is presented as a language for making assumptions explicit. Worked examples are deliberately small enough to check by hand; the reader should build confidence on these examples before trusting larger software systems.

Editorial status. This PDF is an introductory manuscript, not the requested complete Volume I. Several chapters remain substantially shorter than the specified minimum of 20,000 words each, and Chapter 1 still requires technical and editorial review. Its references are recommended starting points, not a verified comprehensive bibliography, and it has not undergone independent academic, technical, or publication review. It should not be represented as a final encyclopedia volume.

How to use this volume

Read the first two chapters together: an intelligent agent is an abstract decision-maker, while a robot must also obey physical limits. Chapter 3 introduces the data and evaluation ideas used by Chapters 4 and 5. Chapter 6 can be studied alongside Chapter 2. Keep a notebook of assumptions, units, expected results, and sources. Recalculate each worked example before changing the numbers. A computer can run an experiment quickly, but it cannot decide whether the experiment answers a useful question.

A suitable background is elementary algebra and comfort with simple Python. Where calculus appears, the meaning of the result is explained in words. Experiments described here are illustrative; physical work requires supervision, a risk assessment, an appropriate power supply, guarding, and a tested emergency-stop arrangement. Software examples alone are not a safety system.

1. Introduction to Artificial Intelligence

1. The scope and history of artificial intelligence

What makes a system intelligent?

A thermostat and a route planner both respond to information, but they work at different levels of complexity. The thermostat may compare a measured temperature with a target and switch a heater. A route planner considers many possible sequences of actions before choosing one. Artificial intelligence (AI) studies and builds computational systems that perform tasks associated with perception, reasoning, learning, language, or decision-making. No single definition settles every debate about intelligence. For engineering, the useful question is narrower: what task must the system perform, under which conditions, and how will success be measured?

Modern AI has roots in formal logic, statistics, computing, neuroscience, and control. Early research explored symbolic reasoning and search. Expert systems later encoded specialist rules; statistical methods and machine learning became increasingly important as data and computing resources grew. Contemporary systems combine these traditions in different proportions. A system that performs one task well is not thereby guaranteed to possess general human capabilities.

The boundary between AI and other fields is not fixed. A chess program, a voice interface, and a system that estimates a machine's condition all manipulate information, but they use different representations and are judged by different outcomes. Some definitions emphasize thinking like a person; others emphasize selecting actions that achieve a goal. This book emphasizes the engineering view without assuming that good task performance establishes consciousness, human understanding, or moral agency.

It is equally important not to equate automation with learning. A scheduled irrigation controller can execute a fixed program without modifying it from experience. A learned irrigation predictor may estimate water demand from past observations, but its predictions still need to be translated into commands within equipment and environmental limits. The distinction is useful when diagnosing failure: a fixed rule may be inappropriate, a prediction may be inaccurate, or the system may have acted correctly on a faulty measurement.

A meaningful claim names the task and the comparison. “The model is intelligent” provides little guidance. “On a held-out collection of photographs from the target farm, the model detected 85 of 100 labeled damaged leaves at the chosen threshold” is more informative, provided the collection, labels, and uncertainty are also described. It is still not a claim that the model will perform at that rate in every season or location.

A system should also be distinguished from the model inside it. A prediction model accepts specified inputs and produces an output, such as a number or a category. The surrounding system acquires the inputs, decides when to invoke the model, displays or acts on its output, stores records, and handles exceptions. A model can achieve good results on a curated dataset while the system fails because an image arrives late or because a user interprets the output as a verified diagnosis. Evaluating the model is necessary, but it is not the same as evaluating the service.

Capability and reliability answer different questions. A system may be capable of translating a particular sentence in a demonstration, yet fail on a different dialect or a sentence with a similar but important negation. To describe capability, specify which class of inputs and outputs is under consideration. To describe reliability, identify a distribution of use, a criterion for success, and the uncertainty in the estimate. Neither an impressive example nor a single error rate captures every operational risk.

A short history of ideas

Modern AI did not begin with a single discovery. Its intellectual history includes mechanical calculation, mathematical logic, probability, cybernetics, and electronic computing. In 1950 Alan Turing proposed the imitation game as a way to discuss machine intelligence without first settling the difficult question of whether a machine can think. His paper is a historical argument, not a complete engineering test: a system that produces convincing conversation may still be unreliable, and many useful intelligent systems do not engage in conversation at all (Turing, 1950).

The Dartmouth summer research project of 1956 helped establish “artificial intelligence” as the name of a research field. Early programs demonstrated symbolic problem solving in constrained domains. Progress in a carefully defined game or mathematical puzzle, however, did not automatically transfer to a noisy, open-ended physical environment. Later expert systems represented specialist knowledge as rules, but acquiring and maintaining large rule bases proved difficult. Periods of optimism were followed by reduced investment when performance and expectations diverged.

Statistical learning shifted attention towards methods that estimate patterns from examples. Increased computing power and larger datasets supported advances in neural networks and, later, systems trained on very large quantities of data. The history is not a straight line in which one idea permanently replaces another. A practical robot may combine geometry, symbolic rules, learned perception, probabilistic estimation, and classical control. The most recent component is not necessarily the most suitable one for a particular requirement (Russell and Norvig, 2021).

A historical account also needs to distinguish a research demonstration from a deployable product. A program designed for a controlled laboratory can be an important scientific achievement without being reliable in a changing workplace. A proof of concept may use carefully selected inputs, a knowledgeable operator, and substantial offline preparation. Later engineering has to address

maintenance, unusual inputs, user training, and failure recovery. These activities are less visible than a public demonstration but are central to whether the system is useful.

Changes in available data matter as much as changes in algorithms. A model learned from digitized records reflects the institutions that created those records and the practices used to collect them. If a hospital changed its diagnosis protocol halfway through a dataset, for example, the label assigned to an unchanged condition might change. An apparent improvement or deterioration in a model may then reflect a change in measurement, not a change in the population. Historical context is therefore part of technical interpretation.

Knowledge, uncertainty, and learning

A rule such as “if the charging cable is attached, do not drive” makes a design constraint explicit. Symbolic rules are inspectable, but a rule base can become difficult to maintain and may not cover unexpected circumstances. Probability provides a different tool: it expresses uncertainty about events under a model and available evidence. For instance, if a diagnostic test has a false-positive rate, a positive result is not automatically proof of a fault. Base rates matter as well as test accuracy.

Learning systems adjust a model using data or experience. Supervised learning uses examples with target labels; unsupervised learning looks for structure without those labels; reinforcement learning studies decisions informed by rewards and consequences. Each choice implies a different source of feedback. Learning can complement rules, but a learned prediction must be evaluated for the population and operating conditions in which it will be used.

2. Tasks, agents, and environments

Specifying a task before building a model

A useful specification separates the desired outcome from a particular implementation. Imagine a cart that carries sealed samples between rooms in a building. “Use a neural network” is not a requirement; “transport the samples without entering restricted areas” is closer to one. Even that statement needs operational detail: which rooms, what obstacles, what loads, and which humans are expected to share the route?

For this cart, an agent description can be organized as performance, environment, actuators, and sensors. Performance might include successful deliveries, elapsed time, and unsafe events. The environment includes corridors, doors, pedestrians, and floors. Actuators include drive motors and a warning indicator. Sensors could include wheel encoders and a range sensor. This inventory is useful because a desired behavior cannot be implemented if the relevant information is unavailable or the actuator cannot carry it out. A map is not a sensor: it is a representation that can become outdated.

A rational action is an action expected to perform well according to the stated measure, given the information and resources available. Rationality does not mean omniscience. A cart cannot avoid an obstruction that it cannot sense and has no reason to predict; the designer must therefore specify a response to limited visibility, such as slowing or stopping within a validated operational envelope. Expected performance alone must not be used to justify exceeding hard safety constraints.

A requirement is more useful when an independent reviewer can determine whether it has been met. For a delivery cart, “operate quickly” could be refined into a distribution of delivery times on defined routes, at defined loads and traffic levels. “Be safe” calls for a hazard analysis, protective measures, and evidence appropriate to the machine; it cannot be reduced to a single average delivery

time. Some requirements describe normal behavior, while others describe what the system must do when a component fails.

Consider three related but distinct quantities: a design objective, an operational constraint, and an observation. Short travel time is an objective. A prohibition against entering an occupied restricted zone is a constraint. The value reported by a door sensor is an observation. An optimizer might minimize time among routes that satisfy its model of the constraint, but that does not establish that the model and sensor correctly represent the actual zone. The requirement must extend to monitoring and handling disagreements between the map and the real environment.

Stakeholders can disagree about success. A laboratory technician may want prompt deliveries, a maintenance engineer may prefer equipment that is easy to diagnose, and a person using a wheelchair may need unimpeded passage. Interviews and observation can expose such differences before they become design defects. Document whose needs were considered, who approved the requirements, and which groups were not represented. A technical specification is not neutral merely because it contains numbers.

An operating domain defines the circumstances in which a system is intended and evaluated to work. For a sample cart, it could specify certain buildings, floor conditions, loads, speeds, and times of operation. It should also describe exclusions, such as stairs or corridors undergoing construction. A system that encounters conditions outside that domain needs a defined response: requesting assistance, pausing, or transferring responsibility to a trained operator may be appropriate, depending on the hazard analysis. A list of exclusions is not a substitute for a mechanism that can recognize or otherwise manage relevant boundary conditions.

Requirements often conflict. A narrow cart may pass through more doorways but leave less room for battery capacity; faster travel may increase throughput but reduce the time available to respond to an obstacle. A trade-off analysis makes these tensions visible instead

of pretending that a single number captures them all. Separate a hard limit from a preference. For example, one may prefer a shorter route among routes already shown to satisfy applicable restrictions. A large numerical penalty for a prohibited route is not always equivalent to forbidding it: if the predicted benefit becomes sufficiently large, an unconstrained optimizer may still select the route.

Traceability connects each requirement to design choices and evidence. A simple requirements record can contain an identifier, the requirement's wording, its rationale, the method of verification, its owner, and the result. For the requirement that a cart remain within an authorized route, the record might identify the source of the route map, the enforcement mechanism, test cases for boundary changes, and the person responsible for updating permissions. A failing test should lead to a documented disposition rather than quietly disappearing from the final report.

Agents and environments

An agent receives observations and selects actions. Its environment includes everything relevant to the consequences of those actions. A warehouse robot, for example, observes ranges and images, commands wheel motors, and works among shelves and people. The engineer must decide what to measure: completed deliveries, collision rate, energy use, or a weighted combination. A performance measure that rewards speed alone can encourage unsafe behavior.



Figure 1.1. An agent interacts with its environment through sensing and action; observations and effects are not assumed to be perfect.

An environment is fully observable only when the information available to the agent is sufficient to determine the relevant state. A camera blocked by a box makes an aisle partially observable. The world may also be stochastic: the same motor command can lead to different results when floor friction changes. An agent's internal estimate of the situation is therefore not necessarily the situation itself. Confidence in an estimate should not be confused with proof that it is correct.

Properties of environments

An environment may be deterministic in a model yet uncertain in practice. If a model says a move always advances exactly one grid square, its transition is deterministic. On a real floor, slip and motor variability mean the same command may produce different displacements. A task is episodic when the consequence and evaluation of one decision do not depend on earlier decisions; sorting independent photos can approximate this. Navigation is sequential because an earlier turn changes later opportunities.

A static environment does not change while an agent decides; a dynamic one may. A turn-based puzzle is often treated as static during a player's turn. A corridor with pedestrians is dynamic. Discrete models contain distinct states and actions, whereas real position and time are often represented continuously. Discretizing a corridor into cells simplifies planning, but a large cell can hide a narrow obstacle. The discretization is an engineering approximation that must be checked against the system's purpose.

A useful further distinction is whether the rules of the environment are known. An agent may know the set of permitted moves in a game even though it does not know which move an opponent will choose. A robot in an unfamiliar building may not know how a particular door responds to a command. Unknown dynamics invite experimentation or identification, but physical experiments can cause harm: uncertainty about an outcome is not permission to try it without a risk assessment.

The relevant state is the information required by the chosen model to predict possible next states and evaluate actions. If delivery time depends on remaining battery energy, a state description containing only position can be inadequate. Two carts at the same location can have different available actions if one is nearly discharged. Adding battery charge to the state makes the model more informative, at the cost of a larger state space and a need for a reliable estimate of charge. There is no universally best representation; there is a representation fit for a stated question.

An environment can contain multiple agents with different objectives and information. A pedestrian and a delivery cart share a corridor, but the pedestrian may not know the cart's destination and the cart may not know the pedestrian's intentions. A model that assumes every person will move aside may be convenient, but it is not a sound basis for a safety claim. The system needs a behavior for the possibility that the person does not respond as expected, and the design must be tested with appropriate safeguards before human interaction.

Observability is a property of the task model and information available to the agent, not just the number of sensors. A camera and range sensor can both view the same side of a box while the far side remains hidden. Adding a second sensor may reduce some uncertainty but can introduce correlated errors if both depend on the same obstructed view. Describing the limitations of each observation helps avoid treating a sensor fusion output as independent confirmation when its inputs share a cause of failure.

Reactive, model-based, and goal-directed behavior

A simple reactive policy maps the current observation to an action. A rule such as “stop when the measured distance is below a threshold” can act quickly, but its success depends on sensor coverage, latency, braking distance, and the validity of the measurement. It should not be treated as a complete safety design. If a sensor returns an invalid value, the policy needs an explicit response instead of interpreting the value as clear space.

A model-based agent maintains an internal state derived from observations and previous actions. It might remember that a door was closed even when the door is temporarily outside the camera view. Such memory is useful, but it can become stale; a later observation or a time limit should prompt revision. A goal-directed agent compares possible future actions against a desired state. A utility-based agent can additionally rank outcomes, for example by balancing travel time and energy use within fixed safety boundaries. These architectural categories describe design choices; actual systems may combine them.

A layered architecture can give different responsibilities to different processes. A slow planner may select a sequence of route segments while a faster controller tracks the next segment. A separate monitor may check sensor health and request a stop when a defined condition is violated. This arrangement can improve clarity, but merely drawing separate boxes does not guarantee independence. If the planner and monitor both rely on the same failed sensor, their apparent redundancy may not protect against that failure.

An internal estimate also has a lifetime. Suppose a cart observed that a door was open five minutes ago. A plan based solely on that stored fact can be invalid if someone has closed the door. A timestamp and a rule for refreshing information are therefore as important as the label “open.” The acceptable age of information depends on how quickly the environment can change and on the consequences of acting on an outdated belief. A design should define the response when an update does not arrive.

One design technique is to give a controller an explicit set of modes. An illustrative cart may be in waiting, planning, travelling, paused, or fault mode. A transition from travelling to paused could be triggered by a relevant stop request; resumption may require that defined conditions have been checked. Mode diagrams make it possible to ask what happens when events arrive in an unexpected order. They also expose missing transitions, such as what to do if a fault occurs while the cart is already paused.

Mode names do not establish the safety of an implementation. The actual behavior depends on timing, priority, communication, power, and hardware. If two components issue incompatible commands, the system needs an arbitration rule. If the operating procedure calls for stopping, the design must define and validate what stopping means for that machine and hazard. A general architectural diagram cannot substitute for a safety-rated design or an assessment of the physical installation.

3. Representing knowledge and reasoning

Representations: choosing what the machine can express

An AI program cannot reason directly about every aspect of the world. It works with representations: symbols, numbers, graphs, images, or other data structures that stand for selected features of a situation. The choice of representation determines which questions are easy to ask and which distinctions are lost. An address in a delivery database identifies a destination; a geometric map describes where that destination lies; an access-control rule describes whether a particular cart may enter. These representations answer related but different questions.

A symbolic representation can make relationships explicit. For example, Connected(A, B) records a directed connection between two locations, while Restricted(B) records a property of B. A rule can combine them: if B is restricted and the cart lacks permission for B, do not plan an entry into B. The usefulness of the rule depends on the meanings of “restricted” and “permission,” the correctness of the stored facts, and the point at which the rule is enforced. A label alone cannot guarantee any of those things.

A numerical representation can capture quantities that symbols alone do not express conveniently. A robot's position estimate might be a pair (x, y) measured in metres relative to a specified origin, accompanied by a measure of uncertainty. A feature vector can combine several measured quantities, such as temperature, vibration, and current. Each component needs a definition and

units. If one component is in degrees Celsius and another in milliamperes, their numerical magnitudes cannot be compared meaningfully without considering their scales and the purpose of the comparison.

A representation is an abstraction, not a copy of the entire world. An occupancy grid labels portions of space as free, occupied, or unknown, often with associated probabilities. Its resolution controls the detail it can express. A small object may not be represented at a coarse resolution, while a fine grid increases storage and computation. Changing the representation can therefore change the behavior of an algorithm even when the algorithm itself is unchanged. A planner operating on a grid of point-like cells still needs a method that accounts for the robot's physical size and uncertainty.

Representations also carry social assumptions. A language system trained only on one spelling standard can treat other legitimate spellings as errors. A dataset with only two categories for a complex condition may force ambiguous cases into labels that do not reflect the relevant domain. The designer should ask what has been excluded, whether an “unknown” category is meaningful, and who has authority to revise the scheme. Adding an unknown label is helpful only if the system has a defined response when that label is used.

Logic and explicit knowledge

Propositional logic represents statements that are either true or false in a particular interpretation. Let C mean “the charging cable is connected” and D mean “the robot is permitted to drive.” The rule $C \rightarrow \neg D$ reads “if C, then not D.” If C is known to be true and the rule is accepted, the inference rule modus ponens gives $\neg D$. The symbols express the logical structure; sensors and software must still establish whether C is true in the physical installation.

The converse does not follow: $\neg D$ does not imply C. The robot might be prohibited from driving for another reason, such as an open safety gate. Confusing a rule with its converse creates unsafe

diagnoses. It is also important to distinguish “not known to be connected” from “known not to be connected.” An absent or failed cable sensor is not evidence that the cable has been removed. A conservative design must define what to do with unknown or inconsistent inputs.

First-order logic can express relationships between objects, for example that every room designated restricted requires authorization. A knowledge base contains such statements; an inference procedure derives consequences from them. A sound inference procedure derives only conclusions supported by the premises under its formal semantics. Sound reasoning cannot repair a false premise or an incomplete world model. Representations must be updated, checked against evidence, and assigned clear ownership.

The distinction between syntax and semantics helps prevent mistakes. Syntax specifies which strings are valid expressions. Semantics specifies which interpretations make an expression true. A fact stored as `door_open` has a meaning only if the system defines which door, at what time, and according to which observation. In a multi-door building, a relation such as Open(door_1, time_12) carries more of the context needed to interpret the fact.

Rules also need a strategy for conflict and change. Imagine one database reports that a door is open while a newer sensor report says it is closed. Classical logic does not by itself decide which source is newer or more reliable. The system must record provenance and time, apply a documented update policy, and treat unresolved disagreement as uncertainty. Overwriting old records without a trace may make an incident difficult to investigate.

Knowledge graphs and ontologies organize entities and relations, such as rooms, doors, and permitted routes. They can help different modules use consistent names, but sharing a label does not guarantee agreement on its meaning. A “clear corridor” label might describe a corridor free of boxes while omitting people, temporary spills, or the robot's own stopping distance. Definitions should be

tied to the intended use, and missing relationships should not automatically be treated as false in every representation.

Truth tables make small logical claims testable. For propositions P and Q, the implication $P \rightarrow Q$ is false only when P is true and Q is false; it is true in the other three combinations. Thus an implication alone says nothing about whether Q holds when P is false. In the charging example, if the cable is disconnected, the rule “if connected, do not drive” does not establish permission to drive. Other conditions, including a closed guard and a valid route, may still be required.

Conjunction and disjunction provide building blocks for more specific rules. $P \wedge Q$ is true when both P and Q are true; $P \vee Q$ is true when at least one is true, including the case in which both are true. The negation $\neg P$ is true when P is false in the chosen interpretation. A specification that says “the gate is closed or the robot is stopped” permits both to be true. If the intention is exactly one, an additional restriction is needed. Writing examples of allowed and disallowed states is a practical way to uncover ambiguous requirements.

A rule base can be tested with cases. Given CableConnected = true and a rule CableConnected → DriveInhibited, an inference engine should derive DriveInhibited. Given CableConnected = false and no other facts, it should not derive permission to drive solely from that rule. Given conflicting measurements, the test should establish how the implementation represents disagreement. The mathematical rule does not specify whether to trust an electrical interlock, a network message, or a user's statement; that is a separate system-design decision.

An inference is valid when its conclusion follows from its premises in every interpretation permitted by the logic. For example, the premises $P \rightarrow Q$ and P entail Q. By contrast, $P \rightarrow Q$ and Q do not entail P: Q might be true for another reason. A counterexample establishes the invalidity of this second argument. Let P be false and Q be true; the implication and Q are both true, while P is false.

This simple test helps distinguish a plausible explanation from a logically necessary conclusion.

A second useful inference rule is modus tollens: from P → Q and ¬Q, infer ¬P. This rule is valid in classical propositional logic, but its application depends on the premises. If Q represents a sensor report rather than the physical event the rule was intended to describe, a sensor failure can invalidate the engineering conclusion. Formal validity guarantees the relationship between premises and conclusion, not the truth of the premises.

Consider a maintenance rule: if the battery is disconnected, the motor receives no battery power. A technician observes that the motor receives no battery power. It does not follow that the battery is disconnected; a blown fuse or an open switch could also explain the observation. This distinction between deduction and diagnosis matters in troubleshooting. Diagnosis generally compares several possible causes against observations, often with uncertain evidence and additional tests.

4. Problem solving by search

Search as structured problem solving

A search problem specifies an initial state, possible actions, a transition model, a goal test, and, when relevant, action costs. Consider four locations with edges S–A, S–B, A–G and B–G. If each move costs one unit, breadth-first search explores by increasing number of moves and finds a two-edge route from S to G. It is complete on a finite graph when repeated states are handled, and it gives a shortest path in number of edges when all edge costs are equal. If costs differ, uniform-cost search instead expands the frontier state with lowest accumulated cost.

A* uses the priority f(n) = g(n) + h(n). Here g(n) is the cost of the path found so far to node n, and h(n) estimates the remaining cost to the goal. Suppose g(A) = 3 and h(A) = 2, while g(B) = 2 and h(B) = 5. The priorities are 5 and 7, so A is selected first. A heuristic is admissible when it never overestimates the true remaining cost.

Optimality claims also depend on the algorithm's termination and graph-search rules; for a standard closed-list implementation without reopening nodes, consistency of the heuristic is a common sufficient condition. An estimated straight-line distance can be useful when travel cost is distance and obstacles can only make a route longer.

In a graph, a node represents a state and an edge represents an available transition. A path is an ordered sequence of connected edges. Search constructs or explores part of this graph; it does not need to generate every possible state in advance. The representation determines what the algorithm can distinguish. If two routes reach the same room but leave different battery levels, merging them into one node may discard information needed to reach a charging station safely.

The branching factor is the number of successors generated from a state. In a uniform tree with branching factor b, there are b^d nodes at depth d, where d is the number of edges from the root. If b = 3 and d = 4, there are $3^4 = 81$ nodes at that depth and 1 + 3 + 9 + 27 + 81 = 121 nodes through depth four. This rapid growth explains why a search that is easy on a small diagram may become costly in a realistic problem. Graphs can contain repeated states and cycles, so the tree count is an illustration rather than an exact count for every graph.

A solution's path cost adds its edge costs when the model defines additive costs. A search method may be complete, optimal, both, or neither under specified assumptions. Completeness means it finds a solution if one exists within those assumptions; optimality means the returned solution has the best cost under the modeled objective. Neither property establishes that the chosen model captures every real-world hazard.

Problem formulation influences search performance before an algorithm is selected. An overly detailed state can create a huge number of distinct possibilities, many of which make no difference to the goal. An overly coarse state can merge situations that require

different actions. Suppose a route planner represents a location only by room number. If access depends on whether a door is locked, the model needs a way to represent or query door status. If it also stores the paint color of every wall, the additional detail may increase complexity without helping this planning task.

A goal test and a cost function do different jobs. The goal test decides whether a state meets the stated destination condition. The cost function ranks valid paths. Reaching room G may satisfy the goal, but two paths to G can have different travel times or energy costs. If both time and energy matter, the units and combination rule must be specified. For instance, a cost measured in seconds can include an energy-related penalty only after defining a conversion or a multi-objective method. Calling an arbitrary weighted sum “the best route” conceals a value judgment about those weights.

Some tasks have several acceptable goals. A cart seeking a charger can accept any compatible charging station, not just one named location. A goal test can recognize a set of states, and the search can stop at the appropriate selected goal under the algorithm's assumptions. If availability changes during execution, the planning result must be checked against current conditions; a charger that was free when the route was computed may be occupied on arrival.

Tree search and graph search treat repeated states differently. Tree search can generate a separate node for every path, even if several paths arrive at the same state. Graph search can use a record of visited states or best costs to avoid unnecessary repetition. Which paths may safely be merged depends on the state representation and objective. If future feasibility depends on how much battery remains, two arrivals at the same location with different remaining energy are not equivalent states for that task.

Path constraints also affect representation. If a delivery vehicle is prohibited from carrying food after carrying a certain chemical unless cleaning has been performed, the vehicle's location alone cannot determine whether a food-delivery action is permitted. The

state must include a relevant cleanliness or history condition, and actions must update it. This does not require storing every past event in the search state; it requires retaining the information needed to determine future actions and their consequences.

Comparing search strategies

Breadth-first search (BFS) stores frontier states in a first-in, first-out queue. It expands all states at depth one before states at depth two. In a finite graph, marking each discovered state prevents cycles from repeatedly filling the queue. BFS minimizes the number of edges when all edges have the same positive cost; it does not generally minimize a sum of different costs. Its memory use can become large because an entire frontier may need to be retained.

Depth-first search (DFS) follows one branch before backtracking. A graph implementation can mark visited states to avoid revisiting them. DFS may use less frontier memory in a tree, but its first solution need not be shallow or cheap. In an infinite search space, an unbounded branch can prevent it from finding a solution elsewhere. Depth limits can prevent that behavior but may exclude a solution beyond the limit.

Uniform-cost search (UCS) chooses the frontier entry with the smallest accumulated path cost. When edge costs are nonnegative, and stale entries are discarded, it returns a least-cost path if a goal is reached under the usual finite-graph assumptions. This distinction matters for the sample cart: a route with two doors may be slower or more energy-intensive than a route with three easy corridor segments. Costs should describe what the system actually intends to optimize, rather than being chosen merely because they make the algorithm convenient.

The data structure matters because it determines which frontier entry is examined next. A stack implements last-in, first-out behavior for DFS; a queue implements first-in, first-out behavior for BFS; a priority queue orders UCS entries by accumulated cost. The rule for inserting or updating entries matters as much as the

container. In UCS, marking a state final when it is first generated can reject a cheaper route discovered later. With nonnegative costs, the usual graph-search algorithm finalizes a state when its lowest-cost entry is removed from the queue.

Search can also fail because of resource limits rather than because the goal is unreachable. A planner may exhaust memory, run out of time, or receive an incomplete map. Its interface should distinguish “no path exists in this graph” from “the computation stopped before an answer.” A caller that interprets both as a safely verified absence of obstacles may behave dangerously. The appropriate fallback depends on the task and must itself be assessed.

For an explicitly represented finite graph with V reachable vertices and E examined edges, standard BFS with a queue and a visited set runs in O(V + E) time, assuming constant-time access to adjacency lists and visited entries. The notation describes how the work grows with the size of the represented graph; it does not say exactly how many seconds a particular implementation takes. Storage for the queue and visited information can be O(V). If the graph is generated implicitly, V and E may themselves grow rapidly with the depth of the problem, so the apparently linear graph bound does not remove the exponential growth of a large state space.

Iterative deepening runs depth-limited search repeatedly with increasing limits. In a tree with a finite branching factor, a solution at finite depth is eventually reached if sufficient resources are available. Earlier levels are visited several times, but most nodes in a broadly branching tree occur near the deepest level, so repeated work can be acceptable. This method can combine the shallow-solution behavior of BFS with a smaller depth-first search frontier, subject to the details of cycle handling and the representation. It does not make unequal-cost shortest-path problems equivalent to shortest-depth problems.

Greedy best-first search selects a state using an estimate of remaining distance alone, h(n). It can reach a goal quickly when the estimate is informative, but it may ignore the cost already spent. A*

uses g(n) + h(n), keeping that past cost in view. The difference is not merely a naming convention: a route that looks close to a goal can be expensive to reach. Both methods depend on the model, and a poor heuristic can add computational work or lead a greedy method toward an unfavorable region.

Suppose BFS operates on a finite graph using adjacency lists. It first marks the start discovered and places it in a queue. Each iteration removes the oldest entry, tests for a goal, and appends undiscovered successors. The parent of a newly discovered state is recorded once, at discovery. Because all edges have equal positive cost, no later discovery can produce a path with fewer edges than the first discovery. A path can be reconstructed by following parent links from the goal back to the start.

Now change the model so that different edges have different travel times. The first discovery of a state need not be the cheapest. In the earlier weighted example, B is first reached directly from S at cost 4 but later reached through A at cost 2. Uniform-cost search therefore maintains the best known cost and replaces a more expensive route when it finds a cheaper one. It must not permanently close B merely because B was first generated. This is a difference in algorithmic logic, not just a difference between a queue and a priority queue.

A practical measurement of search performance should state whether graph construction time, file loading, and path reconstruction are included. It should also report how many search problems were attempted and how timeouts were treated. A planner that solves only the easiest half of a benchmark quickly cannot fairly be compared by averaging only its successful runs against a planner that solves every case. Report coverage and resource limits alongside timings.

A weighted route: complete worked trace

Consider directed edges S → A with cost 1, S → B with cost 4, A → G with cost 10, A → B with cost 1, and B → G with cost 1. Here S is the start and G is the goal. The two-edge route S → A → G costs 11, while the three-edge route S → A → B → G costs 3. A breadth-first traversal that checks A before B and returns the first goal can find S → A → G. The fewest-edge objective and the lowest-cost objective are different.

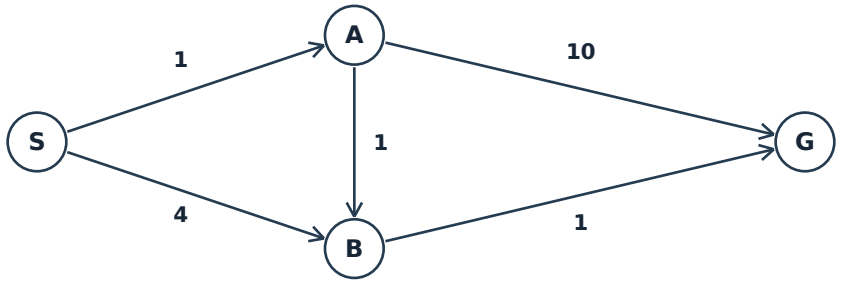


Figure 1.2. Directed weighted graph used in the route example. Arrows show permitted transitions; numbers are edge costs.

UCS begins with S at cost 0. Expanding S creates A at cost 1 and B at cost 4. It next expands A and discovers G at cost 11 and a cheaper route to B at cost 2. Expanding B at cost 2 discovers G at cost 3. When G is removed from the priority queue at cost 3, the algorithm returns S → A → B → G. The older B entry at cost 4 remains in the queue but is stale: its cost no longer matches the best known cost to B. Skipping stale entries avoids using that outdated route.

The following Python program implements the example. It uses a priority queue, a dictionary of best known costs, and parent links. The graph is finite, every edge weight in this example is nonnegative, and a missing route returns None. The counter breaks ties without requiring nodes themselves to be orderable. The assertions are tests for these cases, not a proof of correctness for all possible inputs.

```
from heapq import heappush, heappop
from itertools import count
```

```
def shortest_path(graph, start, goal):
    serial = count()
    queue = [(0, next(serial), start)]
    best = {start: 0}
    parent = {start: None}
    while queue:
        cost, _, node = heappop(queue)
        if cost != best[node]:
            continue  # a cheaper route was found later
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return cost, list(reversed(path))
        for neighbor, weight in graph.get(node, []):
            if weight < 0:
                raise ValueError("negative edge cost")
            candidate = cost + weight
            if candidate < best.get(neighbor, float("inf")):
                best[neighbor] = candidate
                parent[neighbor] = node
                heappush(queue, (candidate, next(serial), neighbor))
    return None

graph = {
    "S": [("A", 1), ("B", 4)],
    "A": [("G", 10), ("B", 1)],
    "B": [("G", 1)],
    "G": [],
}
assert shortest_path(graph, "S", "G") == (3, ["S", "A", "B", "G"])
assert shortest_path(graph, "G", "S") is None
assert shortest_path(graph, "S", "S") == (0, ["S"])
print(shortest_path(graph, "S", "G"))
```

For reproducibility, save the program as `route.py` and run it with Python 3: `python route.py`. The expected output is `(3, ['S', 'A', 'B', 'G'])`. The input graph must have nonnegative edge costs throughout; the defensive check only examines edges reached before the algorithm returns, so it does not validate the entire input. A production implementation should validate the graph up front and define behavior for invalid or non-finite weights. More importantly, a shortest path through an abstract graph is not by itself a collision-free or dynamically feasible route for a physical robot.

Heuristics and the limits of A*

A heuristic supplies a lower bound or an estimate of remaining cost. Let h(G) = 0, h(B) = 1, h(A) = 2, and h(S) = 3 for the weighted graph above. These are the true minimum remaining costs in this example, so they are admissible. A* expands S with f = 0 + 3 = 3. It then sees A with f = 1 + 2 = 3 and B with f = 4 + 1 = 5. Expanding A improves B to g = 2, making f = 3, and produces G at g = 11. Expanding B then finds G at g = 3. Selecting that goal yields the optimal route.

A heuristic h is consistent when $h(n) \leq c(n, n') + h(n')$ for every edge from n to n′, and h(goal) = 0. Here c is that edge's cost. Intuitively, the estimated cost from a node cannot exceed the cost of a single step plus the estimate from its successor. Consistency is convenient for standard graph-search implementations that close expanded nodes; with an admissible but inconsistent heuristic, reopening a node after finding a cheaper path may be required. If h is always zero, A* uses g alone and behaves like UCS. If a heuristic overestimates, it may reduce search effort, but the usual optimality guarantee no longer follows.

For a rectangular grid with only horizontal and vertical unit-cost moves, Manhattan distance is $h = |x - xg| + |y - yg|$. Here (x, y) is the current cell and (xg, yg) is the goal. Each permitted move changes one coordinate by one cell, so at least that many moves are required in an obstacle-free grid. Obstacles that only block moves cannot shorten the shortest route, making the distance an admissible heuristic under these assumptions. At (1, 2) with goal (4, 4), h = 3 + 2 = 5 moves.

The assumptions are essential. If diagonal moves cost one unit, five is not necessarily a lower bound: from (1, 2), two diagonal moves and one horizontal move reach (4, 4) in three moves. Likewise, a geometric distance in metres cannot be added directly to an accumulated cost in seconds without a justified conversion. A useful heuristic must respect both the action model and the units of the cost function.

If a goal is unreachable, a finite graph search can terminate after exhausting reachable states. In a very large or infinite graph, termination needs further assumptions. When costs can be zero or arbitrarily small, claims about how many nodes will be explored require care. Testing a heuristic on a few easy maps cannot establish admissibility; derive the property from the allowed moves and costs, then use tests to find implementation mistakes.

Consider a different graph with edges S → A costing 2, A → G costing 2, S → B costing 1, and B → G costing 10. The optimal route through A costs 4; the route through B costs 11. If h(A) is incorrectly set to 100, h(B) to 0, and h(G) to 0, A* first expands B because its priority is 1 while A's is 102. It then places G on the frontier with priority 11. An implementation that returns when G is removed can return the cost-11 route before considering A. This counterexample shows why overestimation can invalidate the usual optimality guarantee; it does not show that every overestimating heuristic fails on every graph.

A heuristic may be obtained from a relaxed problem. Removing obstacles from a grid cannot increase the shortest route cost, so the unobstructed distance can be a lower bound if moves and costs remain compatible. This principle requires a precise relationship between the relaxed and original problems: every original solution must still be allowed in the relaxation with no greater cost. If the relaxation accidentally removes a shortcut or changes cost units, its solution cost may no longer be a lower bound.

Laboratory: compare two search objectives

This exercise uses an abstract graph, not a physical robot. It compares the shortest path in edges with the least-cost path found by the preceding uniform-cost program. The graph and neighbor order are fixed so that the first BFS route is reproducible. The queue stores nodes waiting to be expanded, and the parent dictionary serves both to mark discovered nodes and to reconstruct the route.

```
from collections import deque
```

```
def breadth_first(graph, start, goal):
    queue = deque([start])
    parent = {start: None}
    while queue:
        node = queue.popleft()
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return list(reversed(path))
        for neighbor, _ in graph.get(node, []):
            if neighbor not in parent:
                parent[neighbor] = node
                queue.append(neighbor)
    return None

graph = {
    "S": [("A", 1), ("B", 4)],
    "A": [("G", 10), ("B", 1)],
    "B": [("G", 1)],
    "G": [],
}
assert breadth_first(graph, "S", "G") == ["S", "A", "G"]
assert breadth_first(graph, "G", "S") is None
assert breadth_first(graph, "S", "S") == ["S"]
print(breadth_first(graph, "S", "G"))
```

Run the code with Python 3 and check that it prints `['S', 'A', 'G']`. The route has two edges and costs 1 + 10 = 11. The UCS route in the preceding example has three edges and costs 3. BFS ignores the numeric edge weights, as the underscore in the loop makes explicit. The graph is directed: reversing the query does not create reverse edges. Add an edge G → S and predict the result of the reverse query before rerunning the test.

To investigate ties, reverse the order of the two successors of S. Both S → A → G and S → B → G have two edges, and the first returned path can change with neighbor order. The number of edges remains minimal in this finite example. Record the graph, order, algorithm, returned path, edge count, and total cost. Do not infer from this one experiment that BFS always returns a costly path; when all edge costs are equal, its shortest-edge solution is also a least-cost solution.

5. Constraints and planning

Constraints, planning, and execution

A constraint satisfaction problem consists of variables, a domain of possible values for each variable, and constraints that restrict their combinations. Suppose three adjacent rooms A, B, and C each need one of two radio channels, 1 or 2. If A differs from B and B differs from C, the assignment A = 1, B = 2, C = 1 satisfies both constraints. If A and C must also differ, two channels are insufficient for the three-room cycle. A solver should report that no assignment exists under the stated domains rather than silently violating a constraint.

Planning extends the idea to sequences of actions. Each action has conditions that must hold before it can be applied and effects that change the modeled state. For a simplified delivery, `open_door` might require that the door is closed and accessible, and have the effect that it is open. A subsequent `move_through` action requires the door to be open. A plan is valid in the model if each action's preconditions hold at the point where it is scheduled and the final state satisfies the goal.

Execution is not the same as planning. A door may jam, a person may enter the passage, or a sensor may report that the state differs from the prediction. A practical system monitors execution, handles failed actions, and replans or stops when assumptions no longer hold. Classical plans often assume a known starting state and predictable effects; those assumptions should be made explicit before the planner is used in a changing environment.

Backtracking is a basic way to solve a constraint problem: assign a value to one variable, check the constraints that can already be evaluated, and continue. If no value leads to a complete assignment, undo the most recent choice and try another. In the three-room cycle, assigning A = 1 forces B = 2 and then C = 1 if B and C must differ. The constraint A ≠ C is violated, so the solver backtracks. Starting with A = 2 produces the symmetric conflict. With the two-value domains exhausted, there is no solution.

Constraint propagation can remove values before a full assignment is tried. For example, once A = 1, the constraint A ≠ B removes 1 from B's domain. If B has only the value 2 left, the constraint B ≠ C removes 2 from C's domain. A remaining conflict can then be detected early. Ordering variables with small remaining domains first can often reduce search effort, although the benefit depends on the problem and implementation.

In planning, an action description can be inspected in the same way as a contract. For an illustrative door action, list the initial conditions, effects, and conditions that remain unchanged. If opening the door requires a key that the robot does not possess, a model omitting the key can produce an apparently valid but unexecutable plan. Conversely, a model that assumes all doors require the same key may reject an available route. Model validation should include both missing and unnecessarily restrictive assumptions.

Constraints may be hard or soft. A hard constraint defines an unacceptable assignment or action, such as a required access restriction in a planning model. A soft constraint records a preference, such as preferring not to disturb a busy corridor. Treating a soft preference as hard can make a feasible task appear impossible; treating a genuinely mandatory restriction as a preference can allow an unacceptable plan. The distinction should be agreed with the people accountable for the operation.

A planning state is a snapshot of facts relevant to the modeled decision. For a toy delivery task, the facts might be AtCart(A), DoorOpen(A,B), and AtPackage(A). A move action from A to B could require AtCart(A) and DoorOpen(A,B), then replace AtCart(A) with AtCart(B). A pickup action could require the cart and package to be at the same location and the gripper to be empty. These actions omit many physical details. A plan that satisfies them is a candidate for further checks, not evidence that the gripper can lift the package or that the route is physically clear.

A plan monitor compares expected and observed conditions at meaningful points. If an action to open a door finishes without an observation that the door is open, the system should not simply assume success because the command was sent. It can retry only when retrying is allowed and safe, seek a different route, or request assistance. The monitor needs explicit timeouts and a policy for an unavailable observation. Failure handling should be exercised in tests, including cases in which the recovery action itself fails.

A constraint solver's answer depends on the information placed in the domains and rules. In a scheduling problem, let variables T1 and T2 be time slots for two maintenance tasks, each with domain {morning, afternoon}. If the same technician must perform both tasks and they cannot overlap, the constraint T1 ≠ T2 admits two assignments. If each task takes an entire day rather than one slot, the stated domains and constraint are inadequate: an apparently valid assignment can still be impossible in practice. The model should be revised and the assumptions documented.

In a dynamic setting, a previously valid assignment can become invalid. A technician can become unavailable, or a robot assigned to a task can lose power. Replanning may need to preserve commitments already made, avoid changing an action in progress, and notify affected people. Finding any mathematically valid schedule is not sufficient if the cost of disrupting an ongoing activity has not been modeled.

Laboratory: testing a constraint model

This laboratory enumerates assignments for the three-room channel problem. It illustrates the difference between a correct solver and a correct model. The program assumes that every listed room has the same finite domain of channels and that each listed edge means that the rooms at its ends must have different channels. It enumerates all assignments, so its running time grows rapidly as rooms or channels are added. It is intended for small, inspectable examples rather than large installations.

```
  from itertools import product
```

```
def assignments(rooms, channels, edges):
    # Check the model before searching for assignments.
    rooms = tuple(rooms)
    channels = tuple(channels)
    edges = tuple(edges)
    if len(set(rooms)) != len(rooms):
        raise ValueError("room names must be unique")
    if len(set(channels)) != len(channels):
        raise ValueError("channels must be unique")
    for left, right in edges:
        if left not in rooms or right not in rooms:
            raise ValueError("unknown room in an edge")
    results = []
    for choice in product(channels, repeat=len(rooms)):
        assignment = dict(zip(rooms, choice))
        if all(assignment[a] != assignment[b] for a, b in edges):
            results.append(assignment)
    return results

rooms = ("A", "B", "C")
chain = (("A", "B"), ("B", "C"))
triangle = chain + (("A", "C"),)
assert len(assignments(rooms, (1, 2), chain)) == 2
assert assignments(rooms, (1, 2), triangle) == []
assert len(assignments(rooms, (1, 2, 3), triangle)) == 6
print(assignments(rooms, (1, 2), chain))
```

Run this program with Python 3. The chain has two assignments: A = 1, B = 2, C = 1 and the reversed assignment A = 2, B = 1, C = 2. With the additional A–C constraint and only two channels, there are no assignments. With three channels, the triangle has six assignments: three choices for A, two remaining choices for B, and one for C. The assertion checks the count; a separate inspection can check the assignments themselves.

The enumeration examines k^n candidate assignments for n rooms and k channels, before accounting for the cost of checking constraints. With ten rooms and three channels it examines 3^{10} = 59,049 candidates; with twenty rooms it examines 3^{20} = 3,486,784,401. A backtracking solver can reject inconsistent partial assignments earlier, but its performance still depends on the problem and variable ordering. The point of the calculation is to anticipate resource growth, not to predict a precise runtime on a particular computer.

Try removing the B–C edge, then predict the number of two-channel assignments before running the program. There are four: A and B must differ, while C can use either channel. Now imagine that a real radio installation also has distance, power, and interference requirements. This toy program does not model them. A valid assignment under the toy constraints is therefore not evidence of an acceptable physical channel plan. Add real constraints only after consulting relevant domain expertise and defining what measurements or calculations support them.

Planning with incomplete and changing information

Classical planning often describes a single known initial state, deterministic actions, and a goal. These assumptions can be useful for understanding a problem, but they do not fit every task. If a cart does not know whether a door is locked, the plan may need a sensing action or a branch for different observations. A contingent plan represents different actions for different outcomes. For example: check the door; if it is confirmed open, continue under the relevant operating rules; if it is confirmed closed, request assistance or use an authorized alternative; if the observation is invalid, follow the defined fault response.

The order of actions can matter even when each action appears individually valid. A cart cannot deliver a book before it has been loaded, and it cannot assume a door has opened merely because an open command was sent. Preconditions express what must hold before an action; effects express what the model predicts afterward. A monitor compares those predictions with observations. If the predicted effect does not occur, continuing the original sequence may compound the error.

Replanning has costs. Recomputing a route every time a noisy reading changes can cause unstable behavior or excessive delay. Ignoring new information for too long can make a plan stale. A design may use thresholds, timeouts, or explicit events to determine when replanning is warranted, but those choices should be evaluated in the intended environment. A route planner cannot

on its own establish that a physical path is safe; it depends on the correctness and freshness of the information supplied to it.

A plan is also a commitment to other participants. If a human is loading a cart, unexpectedly moving the cart because a new route was computed may be unacceptable. The system needs coordination rules that distinguish an editable future plan from an action currently being performed. A good architecture makes the handoff between planning and execution explicit, including cancellation, acknowledgement, and recovery from failed communication.

6. Probability and decisions

Reasoning with uncertain evidence

Probability offers a way to revise beliefs when observations are imperfect. Bayes' rule is P(F | A) = P(A | F)P(F) / P(A), where F is a fault, A is an alarm, P(F) is the prior probability of a fault, and P(F | A) is the probability of a fault after observing the alarm. P(A | F) is the probability of an alarm when the fault is present. The denominator is the overall probability of an alarm across fault and no-fault cases.

Suppose 1% of otherwise comparable machines have a fault, a test alarms on 90% of faulty machines, and it alarms on 5% of machines without the fault. In 10,000 hypothetical machines, approximately 100 have the fault, of which 90 alarm. Of the 9,900 without the fault, 495 alarm. Among the 585 alarms, 90 correspond to faults; therefore P(F | A) = 90/585 ≈ 15.4%. The alarm is informative, but a positive alarm is not a diagnosis. These figures depend on the assumed base rate and test characteristics; if the population or operating conditions change, the answer can change.

A decision also depends on consequences. Inspecting a low-risk alarm may be reasonable even if most alarms are false, whereas an irreversible action based on that alarm may not be. A simple expected-loss comparison can support a decision, but estimated

probabilities and costs may be uncertain, and some safety or legal constraints cannot be traded away as ordinary costs.

The denominator in Bayes' rule can be expanded using the law of total probability: P(A) = P(A | F)P(F) + P(A | not F)P(not F). In the example, the two terms are 0.90 × 0.01 = 0.009 and 0.05 × 0.99 = 0.0495. Their sum is 0.0585, so 0.009/0.0585 is approximately 0.1538. The two routes to the answer—counting a hypothetical population and calculating with probabilities—should agree apart from rounding.

Now suppose the prior fault rate rises to 10%, while the sensitivity and false-positive rate stay at 90% and 5%. Then the posterior after an alarm is 0.09/(0.09 + 0.045), or about 66.7%. The test did not become more accurate; the population being tested changed. In practice, both test properties and the base rate may be estimated imperfectly, so a sensitivity analysis should vary plausible values rather than presenting one number as certain.

Repeated evidence must not automatically be treated as independent. Two alarms may be caused by the same damaged cable or the same environmental condition, so multiplying likelihoods as if each arose independently can exaggerate confidence. A belief state can represent a distribution over possible physical states when the true state is hidden. New observations revise that distribution, while actions and elapsed time can change the predicted state. Such estimates are only as useful as their observation and transition models.

A probability is always attached to a defined event and a set of assumptions. “There is a 20% chance of failure” is ambiguous until failure, the time interval, and the operating conditions are specified. A probability of a failure during one hour of a fixed operating mode cannot be interpreted automatically as the probability over an entire year. If an estimate is based on historical data, the underlying conditions may have changed, and rare failures may be poorly represented.

Prediction uncertainty can also be separated from measurement error. A distance sensor can have uncertainty because of limited resolution and variable reflections; a prediction of where a person will move has additional uncertainty because the person's future behavior is not determined by the current reading alone. Treating both as a single unexplained “confidence score” makes it difficult to decide what evidence would reduce uncertainty. Measurement, state estimation, and prediction should be evaluated as distinct stages when possible.

Suppose an alarm has a posterior fault probability of 0.1538 under the earlier assumptions. If inspection costs 2 units regardless of the result, and ignoring an actual fault has an expected loss of 30 units while ignoring a healthy machine has zero loss, the expected loss of ignoring is about $0.1538 \times 30 = 4.61$ units. Under this deliberately simple model, inspection has lower expected loss. The comparison assumes inspection prevents the stated loss and introduces no other cost or risk. If inspection is imperfect, delayed, or disruptive, those consequences belong in the model. The numerical example illustrates decision analysis, not a real maintenance recommendation.

Conditional probability describes an event relative to another event: $P(A \mid B) = P(A \cap B)/P(B)$, provided $P(B)$ is greater than zero. Here $A \cap B$ means both events occur. The conditioning direction matters. $P(\text{alarm} \mid \text{fault})$ is the sensitivity of the alarm, while $P(\text{fault} \mid \text{alarm})$ is the probability of a fault after an alarm. They are generally different because the frequency of faults and false alarms also affects the result.

Two events are independent when $P(A \cap B) = P(A)P(B)$. They are conditionally independent given a condition C when $P(A \cap B \mid C) = P(A \mid C)P(B \mid C)$, assuming the probabilities involved are defined. This is a modeling assumption to examine, not a default guarantee for repeated measurements. Two cameras may produce correlated errors under glare, even if their electronic noise is otherwise independent. An analysis that assumes independent failures can substantially understate the chance of a shared failure.

Consider a toy system with two alarms, each independently failing to trigger with probability 0.1 when a particular fault is present. Under the stated conditional-independence assumption, the probability that both miss that fault is $0.1 \times 0.1 = 0.01$. If the alarms share a power supply whose failure disables both, this calculation is incomplete: a common-cause failure creates dependence that the multiplication did not include. The example is not evidence that either alarm has an actual miss probability of 0.1, nor that adding a second alarm achieves a required safety level.

Working with uncertainty in software

The numerical Bayes example can be expressed as a small reusable function. Its inputs are the prior probability of a fault, the sensitivity of an alarm, and the false-positive rate. Each must lie between zero and one. The function rejects an impossible conditioning event: if the model assigns probability zero to an alarm, the posterior given that alarm is undefined under this calculation.

```
def fault_given_alarm(prior, sensitivity, false_positive):
    values = (prior, sensitivity, false_positive)
    if not all(0 <= x <= 1 for x in values):
        raise ValueError("probabilities must be in [0, 1]")
    fault_and_alarm = prior * sensitivity
    alarm = fault_and_alarm + (1 - prior) * false_positive
    if alarm == 0:
        raise ValueError("alarm has zero probability")
    return fault_and_alarm / alarm

p = fault_given_alarm(0.01, 0.90, 0.05)
assert abs(p - 90 / 585) < 1e-12
assert abs(fault_given_alarm(0.10, 0.90, 0.05) - 2/3) < 1e-12
try:
    fault_given_alarm(0, 0, 0)
except ValueError:
    pass
else:
    raise AssertionError("zero-probability event was accepted")
print(f"posterior: {p:.4f}")
```

Save this as `alarm.py` and run it with Python 3. The expected output is `posterior: 0.1538`. The validation prevents ordinary out-of-range numerical inputs and the test covers a zero

denominator; it does not establish the quality of the prior or the test characteristics. If the inputs are measured quantities, the uncertainty and context of those measurements should accompany the result. A mathematically exact calculation using an unjustified prior can still lead to an unreliable decision.

Sensitivity exercise. Keep sensitivity at 0.90 and the false-positive rate at 0.05. Calculate the posterior for prior values of 0.01, 0.05, and 0.10. The approximate results are 0.154, 0.486, and 0.667, respectively. For the middle value, the numerator is 0.045 and the denominator is 0.045 + 0.0475 = 0.0925. Explain why the output changes even though the test properties are held fixed. Then ask what evidence would justify each prior in the actual population. Do not treat this sensitivity exercise as a replacement for measuring the test in that population.

Sequential decisions and expected outcomes

An action can be evaluated by considering possible outcomes and their values. In a simplified, one-step example, assume action A succeeds with probability 0.8 and gives utility 10 on success or 0 on failure. Its expected utility is $0.8 \times 10 + 0.2 \times 0 = 8$. Action B succeeds with probability 0.6 and gives utility 12 on success or 2 on failure, yielding $0.6 \times 12 + 0.4 \times 2 = 8$. Both actions have the same expected utility under these assumptions, even though their outcome distributions differ. If a failure of one action can cause unacceptable harm, equal expected utility does not make the actions equally acceptable.

For a sequence of decisions, an action can change the options available later. A Markov decision process (MDP) models a set of states, available actions, transition probabilities, and rewards. The Markov assumption says that, given the current modeled state and action, the distribution of the next state does not depend on earlier history. This is a property of the model; it need not be true of a poorly chosen state description. If a door's future availability depends on whether it was recently opened, a state containing only the cart's position may not satisfy the assumption.

A discounted return is $G = r_0 + \gamma r_1 + \gamma^2 r_2 + \ldots$, where r_0, r_1, and r_2 are rewards received at successive time steps and γ is a discount factor between 0 and 1. For rewards 2, 4, and 6 over a three-step episode with $\gamma = 0.5$, $G = 2 + 0.5(4) + 0.25(6) = 5.5$. Discounting gives earlier rewards greater weight when γ is below 1. It is a modeling choice, not a universal moral principle: a long-term harm should not be ignored merely because a numerical discount makes its contribution small.

When the physical state is partly hidden, the agent can maintain a belief distribution rather than pretending to know one exact state. A partially observable model can describe how observations relate to hidden states and how actions change them. Exact planning over beliefs can be computationally demanding. Approximations may be necessary, but the approximation error and the response to unexpectedly poor observations should be part of the engineering evaluation.

Expected utility is a probability-weighted average under a specified utility model. It is useful for comparing uncertain actions, but it does not mean that the average outcome will actually occur on any single trial. In the earlier action A example, a single outcome is either 10 or 0, not 8. It is also sensitive to the values and probabilities assigned by the analyst. When those inputs are uncertain, reporting a range of results can be more informative than reporting a single preferred action.

The value of additional information can be illustrated with a deliberately simple choice. Suppose a part is faulty with probability 0.1. Replacing it costs 4 units whether or not it is faulty; doing nothing costs 20 units if it is faulty and zero otherwise. Without another observation, the expected cost of doing nothing is 2 units, so it is preferred to replacement under these assumptions. Imagine a perfect, free test: after the test, replace only when the part is faulty. The expected cost is $0.1 \times 4 = 0.4$ units, a reduction of 1.6 units. This is the expected value of perfect information in the toy model. A real test may be costly, delayed, or imperfect, and its

value cannot exceed this idealized comparison under the same model and choices.

A worked decision rule with unequal error costs

A prediction is not automatically an action. Suppose an inspection system estimates p, the probability that an item is faulty given the available evidence. There are two possible actions in a simplified model: inspect or pass. Inspection costs 3 units and is assumed to prevent all loss from a fault. Passing costs 20 units if the item is faulty and zero if it is not. The expected cost of inspection is therefore 3, while the expected cost of passing is 20p. Inspection has lower expected cost when $3 < 20p$, or $p > 0.15$. At $p = 0.15$ the two modeled costs are equal.

Each assumption matters. If inspection can miss a fault, its expected cost includes the cost of a missed fault after inspection. If inspection introduces delay or damages a usable item, that consequence also belongs in the comparison. If the 20-unit value represents only direct repair cost and omits injury or environmental harm, the resulting threshold may be inappropriate. Legal or safety requirements can impose a mandatory inspection independently of this calculation.

The number p must also be meaningful for the decision. A score labeled “0.2” is not automatically a calibrated 20% probability. Calibration concerns the agreement between predicted probabilities and observed event frequencies across appropriate cases. For example, among a sufficiently large and relevant collection of cases assigned probabilities near 0.2, approximately one fifth should experience the event for that range to be well calibrated. A small sample, a changing population, or a narrow subgroup can make the estimate uncertain. Calibration is not the same as discrimination: a model can rank cases well but assign poor probabilities, or be calibrated overall while giving little useful separation between cases.

A threshold chosen from the wrong population or from the final test set can give a misleading impression of performance. Select and

document the decision policy using appropriate development evidence, then evaluate the fixed policy on separate cases that reflect its intended use. Revisit the policy when the costs, population, or operating conditions change. The aim is not to find a universally correct threshold, but to explain and test a decision rule under stated assumptions.

A worked value-function calculation

The value of a state depends on a policy and a model of the future. Consider a tiny discounted MDP with two states, A and B. In A, the only action gives reward 2 and moves to B with certainty. In B, the only action gives reward 4 and stays in B with certainty. Let the discount factor be $\gamma = 0.5$. The value $V(s)$ is the expected discounted sum of future rewards starting from state s under this fixed policy. Since there is only one action in each state, there is no choice to optimize in this example.

At B, the value satisfies $V(B) = 4 + 0.5V(B)$: the immediate reward is 4 and the next state is B again. Subtracting $0.5V(B)$ from both sides gives $0.5V(B) = 4$, so $V(B) = 8$. At A, $V(A) = 2 + 0.5V(B) = 2 + 4 = 6$. The same results follow from the infinite sequences $4 + 2 + 1 + \ldots = 8$ and $2 + 2 + 1 + \ldots = 6$. The series converge because the discount factor has magnitude less than one.

Now suppose A also has an alternative action that gives immediate reward 5 and then moves to B. Its value is $5 + 0.5(8) = 9$, so it is better than the original action under this model. This conclusion depends on the stated rewards, transitions, and discount factor. If the alternative creates an unmodeled hazard or has a different probability of reaching B, the calculation no longer describes the decision. A high computed value should prompt a check of what the model includes, not an assumption that every relevant consequence has been captured.

More generally, for a fixed policy in a finite MDP, a Bellman equation relates the value of a state to the expected immediate reward plus the discounted value of the next state. The equation decomposes a long-term question into local relationships. Solving it

requires a transition model and reward specification, or estimates from data in methods that do not know the model. Chapter 17 develops reinforcement learning and value functions in more detail; this small example shows the idea without requiring a full algorithm.

Decisions involving another agent

In an adversarial, turn-taking game, a minimax procedure evaluates possible moves assuming the opponent chooses a response unfavorable to the current player. Consider a two-level game in which move A permits opponent outcomes valued 3 and 5, while move B permits outcomes valued 2 and 9. If larger values are better for the first player, the opponent can force 3 after A or 2 after B. The minimax choice is A, with guaranteed value 3 under the model. The result depends on the stated evaluation, available moves, and the assumption about the opponent; it is not a prediction that all people will behave adversarially.

Alpha–beta pruning avoids examining some branches that cannot change the minimax decision, provided the same game tree and evaluation are used. In real human–robot collaboration, people are usually not opponents in a zero-sum game. Modeling them as such may produce inappropriate behavior. The point of a model is to represent the decision setting accurately enough for its intended use, not to force every interaction into a familiar algorithm.

The simple minimax example assumes both players share a known game model and that one player's gain is the other's loss. Many real interactions do not satisfy those assumptions. Two carts may both benefit from avoiding a blocked intersection, and a person helping a robot may have goals that are only partly aligned with the robot's route objective. Depending on the task, coordination, negotiation, or a conservative human-aware policy can be a more appropriate model than adversarial play.

Even within a game, a truncated search uses an evaluation function at the boundary instead of waiting for a final result. Its decision can be wrong if the evaluation overlooks a consequence just beyond the

search limit. A deeper search is not automatically sufficient: an inaccurate model or evaluation can persist at any depth. When computation is limited, the engineering question is how much search to perform, how to test the resulting decisions, and what to do when a reliable answer is not available in time.

A game model also specifies what each participant knows and when they act. In a perfect-information game, the players can observe the relevant current game state. A hidden card or an unknown human intention calls for a different representation. A deterministic game tree with known moves is a useful introduction to adversarial search, but a robot in a public setting should not assume that every person's intended route can be read from their current position.

7. Learning from data and experience

Learning from experience: an overview

A learning problem must identify what changes with experience and what counts as improvement. A supervised learner receives examples consisting of inputs and desired outputs. A model trained on labeled photographs might predict whether an item is damaged. The label is not an intrinsic property of the image file: it is a judgment or measurement produced by a labeling procedure, which may be uncertain or inconsistent. Unsupervised methods work without supplied target labels, for example by grouping measurements according to similarity. A cluster's meaning still requires investigation.

Reinforcement learning concerns sequences of interaction. An agent selects an action, the environment changes, and the agent receives a reward signal. In a simplified delivery simulation, a reward could be negative for each time step and positive for reaching the destination. A policy specifies how the agent selects actions from states or observations. Maximizing accumulated reward can produce surprising behavior if the reward omits important requirements; a policy rewarded only for speed may ignore the condition of the cargo. The reward function is an

engineering choice, not a complete description of what society values.

The three categories are not mutually exclusive parts of a single deployment. A robot can use a supervised perception model, an unsupervised tool for exploratory data analysis, and a fixed planner. Before introducing learning, establish a simple baseline. If a transparent rule achieves the requirement with less data and maintenance, a complex learned model may not be justified. Conversely, a fixed rule may be too brittle when conditions vary. Chapter 3 develops training and evaluation in more depth; this section establishes how learning fits within a broader AI system.

A numerical learning example.

Suppose a fixed rule predicts that a sensor value above 5 indicates a blocked passage. On eight labeled examples, it correctly identifies three blocked passages, misses one blocked passage, correctly declares three clear passages, and falsely declares one clear passage blocked. The observed accuracy is $(3 + 3)/8 = 0.75$. Recall for blocked passages is $3/(3 + 1) = 0.75$; precision for a blocked prediction is $3/(3 + 1) = 0.75$. Equal values here are a consequence of these chosen counts, not a general rule.

A different threshold might improve recall at the cost of more false alarms. The choice depends on the effects of a missed blockage, an unnecessary stop, and the role of other protective measures. Eight examples are far too few to establish reliable real-world performance. They are useful as a calculation exercise and as a small software test, not as evidence for deploying a safety function.

Learning requires a definition of what is being optimized. In supervised learning, a loss function measures the disagreement between a prediction and a target according to a chosen rule. Training adjusts model parameters to reduce an aggregate loss on training examples. A lower training loss does not establish better performance on new cases. The model can fit accidental regularities in the collected data, especially when the dataset is

small or unrepresentative. Chapter 3 develops this distinction through training, validation, and test sets.

A target can be measured incorrectly even when the input is accurate. If a quality inspector and a maintenance log use different definitions of “fault,” training on one and evaluating against the other can create an apparent discrepancy. Record the label definition, who applied it, how uncertainty was handled, and whether the labeling procedure changed over time. When disagreement is substantial, a single hard label may conceal useful information about uncertainty.

In reinforcement learning, the reward signal is not necessarily the same as the real objective. If a simulated cart earns reward for reaching a goal, it might learn behavior that exploits an error in the simulator rather than a useful delivery skill. Testing should inspect the behavior itself, the physical assumptions, and relevant constraints. Safe operation of a real system cannot be inferred from a high reward obtained in a simplified simulation.

Features, models, and predictions

A feature is an input variable chosen or computed for a task. In a simple equipment-monitoring example, the features might be average vibration and temperature over the previous minute. A feature is useful only if it can be obtained reliably at the time a prediction is made. A record of a repair performed after a fault was diagnosed may predict that diagnosis extremely well, but using the repair record to predict the fault before diagnosis would leak future information into the model. Such leakage produces an unrealistically favorable evaluation.

A model is a rule or function that maps inputs to an output, possibly with learned parameters. An illustrative linear score can be written $z = w_1x_1 + w_2x_2 + b$. Here x_1 and x_2 are feature values, w_1 and w_2 are their weights, and b is an intercept. The score z is not necessarily a probability. Its interpretation depends on the model and the method used to transform or threshold it. If $x_1 = 2$, $x_2 = 3$, $w_1 = 0.5$, $w_2 = -1$, and $b = 1$, then $z = 0.5(2) - 1(3) + 1 = -1$. The

arithmetic can be checked independently of any claim about whether the features are useful.

The same observed relationship can have more than one explanation. If high motor current is associated with a fault in a training set, the relationship might arise because the fault increases friction, because a heavy load increases both current and wear, or because of a data-collection artifact. A predictive association alone does not identify which explanation is correct. Intervention, domain knowledge, and a study design suited to causal questions may be needed before claiming that changing one variable will change another.

A feature can become unavailable or change meaning after deployment. A sensor may be replaced with a device using a different range, or a data pipeline may switch from seconds to milliseconds. A model that receives the wrong units can return a precise-looking but meaningless output. Input checks, versioned schemas, and tests with known examples can detect some such errors. They do not replace a review of whether the system remains appropriate for its operating environment.

From a rule to a decision tree

A decision tree organizes a sequence of tests. Each internal node asks a question about an input, each branch corresponds to a possible answer, and each leaf gives an output. The tree may be written by an expert or learned from examples. In either case, the questions and outputs need operational definitions. “Is vibration high?” is incomplete unless “vibration,” its measurement interval, and the threshold for “high” are specified.

Consider a fictional triage tree for a machine. The first test asks whether the temperature measurement is valid. If it is not valid, the output is “measurement unavailable; request inspection.” If it is valid, the second test asks whether temperature exceeds 70 degrees Celsius. An answer of yes produces “inspect”; an answer of no produces “no flag from this rule.” The final output is deliberately

not “machine is safe”: other faults may be present and the rule considers only one measurement.

Trace three inputs. A valid reading of 75 degrees reaches the inspect leaf. A valid reading of exactly 70 degrees reaches the no-flag leaf because the question was strictly greater than 70. An invalid reading reaches the inspection-request leaf without comparing a numerical value to the threshold. Boundary cases reveal why the distinction between greater than and greater than or equal to matters. The threshold in this example is invented for instruction and is not a maintenance recommendation.

A learned tree chooses splits using a criterion computed from training data, such as a measure of label impurity. A deep tree can describe the training examples very closely while depending on accidental details. Pruning, constraints on tree growth, and separate validation are among the tools used to control this problem. A simple tree can be easier to inspect than a large network, but interpretability does not guarantee fairness, correct data, or good predictions. Chapter 3 develops classification and evaluation further.

An introduction to neural computation

A computational neuron combines inputs, applies a transformation, and passes a result onward. In a simple model, $z = \Sigma w_i x_i + b$ and $a = \varphi(z)$. The index i identifies an input, x_i is that input, w_i is its weight, b is a bias, φ is an activation function, and a is the resulting activation. For a rectified linear unit, $\varphi(z) = \max(0, z)$: negative scores become zero and positive scores pass through unchanged. If $z = -1$, the output is 0; if $z = 3$, the output is 3.

A layer contains several such computations. A neural network composes layers, allowing the final output to depend on intermediate representations. Training adjusts parameters using a selected loss and an optimization method. In many modern networks, gradients are calculated by applying the chain rule through the operations in the network; the resulting parameter updates aim to reduce training loss. This description does not imply

that every network uses the same architecture, activation, loss, or training procedure.

A small network may learn a useful mapping, but its output depends on the data, objective, and training process. More layers or parameters do not automatically make a system appropriate for a task. A larger model can require more computation and can still fail on inputs outside the conditions represented in its evaluation. For a first-year introduction, it is enough to understand the roles of input, parameter, activation, loss, and update. Chapter 11 examines neural networks in detail, while Chapter 22 develops deep learning architectures.

Neural terminology is inspired in part by biological ideas, but an artificial neuron is a mathematical operation, not a full model of a biological cell. Analogies can help an initial explanation, yet they should not be used as evidence that a network reasons, feels, or learns in the same way as a person. The correct engineering question remains whether the system meets a defined requirement under tested conditions.

Causal questions and predictive questions

A prediction asks what outcome is likely for a case with given observed characteristics. A causal question asks what would change under an intervention, relative to what would have happened without that intervention. These are different questions. A model may predict that machines with frequent inspections have more recorded faults. It would be a mistake to conclude from that association alone that inspection causes faults: machines already suspected of problems may be inspected more often, and inspection can reveal faults that would otherwise remain unrecorded.

A confounder is a factor that affects both the exposure or intervention of interest and the outcome, creating a potentially misleading association. In the inspection example, machine age could influence both inspection frequency and fault risk. Comparing only frequently and infrequently inspected machines without considering age might mix the effect of inspection with differences

between the machines. This example is illustrative; identifying actual confounders requires knowledge of the process and evidence.

Random assignment can help make groups comparable for a proposed intervention, but it is not always ethical, safe, or practical. Observational studies can also be informative, provided their assumptions and limitations are examined. A predictive model's feature weights do not by themselves establish causal effects. If the intended decision is to change a maintenance policy, the evidence needed for that decision may be different from the evidence needed merely to rank machines for further investigation.

There is a further feedback problem when a prediction changes what is observed. If only flagged machines are inspected, faults among unflagged machines may remain unknown. Future training data may then reflect the earlier model's decisions as well as the underlying fault process. A monitoring plan should consider how to obtain information about unflagged cases without creating unacceptable risk. The appropriate sampling and review procedure belongs to the application's responsible specialists.

Data provenance and the lifecycle of a model

Data provenance is the record of where data came from and how it was changed. For a collection of sensor readings, provenance may include the device, installation, time, calibration status, sampling rate, preprocessing, and any corrections. For labeled photographs, it may include the collection conditions, the labeling guide, the reviewers, and the treatment of ambiguous cases. These records help an investigator decide whether an example is suitable for a task and reproduce the analysis.

Preprocessing can change the question being answered. Replacing missing values with a mean may make a dataset usable by a particular algorithm, but it can also hide a pattern of sensor failure. Normalizing a measurement using statistics from the entire dataset before splitting can leak information from the test set. Even removing duplicate records requires care: two similar images may

be true repeated measurements, separate views of the same object, or accidental copies. The right treatment depends on the experimental unit and the intended claim.

A model lifecycle includes development, evaluation, release, monitoring, and retirement or replacement. A model card or similar record can describe the intended use, known limitations, evaluation data, performance measures, and prohibited uses. Such a document is a communication aid, not a substitute for testing or governance. It should be updated when the model or operating conditions change and should identify who owns that update.

Retirement deserves planning. A model may become unsuitable because a sensor changes, the environment shifts, a security issue is found, or a better method is validated. The system should have an approved fallback that does not silently continue relying on a withdrawn model. The fallback might involve suspending automation or transferring work to trained staff; which option is appropriate depends on the application and its safety requirements.

8. Generative systems

Generative systems and grounded answers

A generative model produces new outputs according to patterns learned from data. Depending on the model and task, an output may be text, an image, audio, or another representation. Generation is distinct from retrieval: a generator can construct a plausible sentence without locating a source that verifies it. A retrieval component can supply relevant documents, but retrieved text can itself be outdated, misleading, incomplete, or irrelevant. A citation is useful only if it actually supports the claim for which it is given.

Consider a service that answers questions about an organization's equipment manual. A system might first retrieve passages, then compose a response based on those passages. An evaluation should test whether the retrieved manual is the correct version, whether the response accurately reflects it, and whether the system

acknowledges when the manual does not answer the question. If a question concerns an urgent physical hazard, an inappropriate answer or an omitted warning can matter more than a fluent style. The service should have a defined boundary and escalation procedure rather than treating all questions as equally safe.

A prompt is an instruction or input provided to a language model. A system may also process content from documents or websites that contains instructions intended to redirect it. Treating all retrieved text as an authorized instruction can create a prompt-injection risk. A sound design distinguishes instructions from data, limits the actions available to the model, checks authorization outside the model, and treats external content as untrusted. These measures reduce some risks but do not guarantee immunity to every attack.

Generated code deserves the same scrutiny as code written by any other means. It can contain incorrect logic, unsafe assumptions, unavailable libraries, or vulnerabilities. Before use, a developer should understand the intended behavior, inspect dependencies and permissions, run relevant tests, and review failure cases. A successful run on a single example does not establish correctness or security.

9. Evaluation and experimental design

Evaluating an AI system as a system

A benchmark score is evidence about performance on a particular dataset and metric, not a guarantee of deployment performance. A responsible evaluation begins with a clearly defined population, operating conditions, and failure consequences. Separate development data from the final evaluation; preserve a record of the data collection procedure and label definitions. Include realistic changes in lighting, location, users, or load if those changes are expected in use. Measure latency and resource consumption as well as prediction quality when decisions must arrive on time.

Errors should be examined in context. For the sample cart, an incorrect estimate of an empty corridor may be more consequential

than an unnecessary pause. Report the number of trials and the distribution of operating conditions, not only a percentage. Where rare events matter, a small test with no observed failures does not demonstrate that failure is impossible. Record uncertainty and the limits of the test, and establish who can suspend operation when performance degrades.

A useful evaluation separates verification from validation. Verification asks whether a component conforms to its specification: does the route algorithm return the intended path on known graph examples? Validation asks whether the resulting system is suitable for the intended use: do the graph, measurements, and operating procedures represent the actual delivery problem well enough? A correct implementation can pass verification and still fail validation because its requirements or environment model are wrong.

Test cases should include ordinary operation, boundary conditions, and fault conditions. For the route program, these include a start equal to the goal, an unreachable goal, multiple routes of different cost, and repeated entries for a state. For a larger system they might include a stale map, a delayed sensor, or a blocked doorway. Simulation is valuable for repeatability and exploration, but a simulator shares the limitations of its models. Physical testing, where appropriate, requires its own controls and cannot be replaced by a simulated success rate.

Suppose an evaluator records 90 successful trials out of 100. The observed success proportion is 0.90; it does not imply that the true probability is exactly 0.90. Trials may be correlated—for example, ten runs through the same corridor on the same day may share a lighting condition. Reporting ten identical repeats as if they represented ten independent buildings would be misleading. Describe the sampling unit, operating conditions, and failures. A confidence interval based on independent, identically distributed Bernoulli trials is not justified automatically when those assumptions do not hold.

Monitoring after deployment should have a responsible owner and a response plan. Track whether inputs remain within the tested operating domain, whether failures or complaints increase, and whether sensors or policies change. A drift alarm is a prompt for investigation, not proof of a particular cause. Version the model and data-processing pipeline, document updates, and preserve the ability to suspend use or return to a previously validated configuration when appropriate.

Evaluation needs a comparison. A baseline might be a fixed rule, an existing manual workflow, or a simpler model trained on the same permitted data. A new system should be assessed against the baseline on comparable cases, with the same outcome definition. If a new classifier has higher overall accuracy but misses more critical cases, the improvement may not serve the application. Compare error types, operating cost, availability, and the burden placed on people as well as the headline score.

A test set is not a renewable source of unbiased evidence. If the team repeatedly checks the test set and changes the model in response, information from that set has influenced development. A fresh evaluation may be needed to support a final claim. When collecting new data is difficult, a protocol can separate model selection from a pre-specified final analysis; the report should disclose deviations from that protocol and limits of the available sample.

Document not only a result but also its denominator. If five missed obstacles occurred in 500 approaches, the observed miss proportion is 1%; if the 500 approaches involved only one type of obstacle on one floor, the result describes that limited experiment. It does not establish the rate in every hallway. Where subgroup results are reported, small denominators can make estimates unstable. Suppressing the denominator or rounding an estimate to an impressive-looking number hides this limitation rather than solving it.

A prospective evaluation is planned before the system is used on the cases being evaluated. Its protocol specifies the population, outcome, test procedure, metrics, and analysis in advance. A retrospective evaluation uses data already collected. Both can be informative, but retrospective data may omit cases that never entered the record and may contain labels influenced by earlier decisions. The choice of evaluation design should follow the question and available evidence, with limitations stated openly.

Suppose a fault predictor is evaluated only on machines that technicians chose to inspect. If inspection was already more likely for machines with an alarm, that dataset is not necessarily representative of all machines. The observed fault frequency among inspected machines can differ from the frequency in the entire fleet. A better study may require a sampling plan that includes machines regardless of the predictor's output, subject to ethical, safety, and practical constraints. This is a study-design issue, not something that can always be fixed by choosing a different classifier.

Designing experiments that can reveal failure

A useful test is designed to challenge a claim. If a team claims that a vision system detects objects on a cart, an experiment should include the objects, backgrounds, lighting, orientations, and occlusions that matter for the intended use. Testing only the images used to choose the model does not measure generalization. Testing only easy examples can also conceal systematic weaknesses even if the examples were technically held out.

Define the outcome before collecting or inspecting the results. For a detector, the outcome might be whether a reference object is correctly identified within a specified time. A reference label needs its own procedure, including how to resolve disagreement. If a system produces no answer before a deadline, decide in advance whether that is a failure, a separate timeout category, or both for different metrics. Removing slow or unsuccessful cases after seeing the results can make performance appear better than it is.

Controls and comparisons help identify causes. If image quality changes with lighting, collect comparable examples under several lighting conditions while holding other relevant factors as constant as practical. Then change one factor at a time when a controlled experiment is feasible. In field data, several factors may change together, so an observed association with lighting is not necessarily proof that lighting caused the error. The report should distinguish a controlled result from an exploratory observation.

Reproducibility requires enough information to repeat the procedure: data selection, software and model versions, configuration, random seeds where relevant, hardware, and the analysis code. A fixed seed can reproduce one random sequence in a specified environment, but it does not show that results are robust to other random choices. Repeating a training process across several seeds can expose variability, provided the design still protects the final test data from repeated model selection.

Finally, a test must be safe and proportionate. A simulated experiment can explore dangerous scenarios without deliberately exposing people to harm, but it may omit important physical effects. Hardware experiments require appropriate safeguards, supervision, and a documented stop procedure. A test that creates an unacceptable risk cannot be justified merely because it would produce useful data.

A test of a full workflow can use a fault-injection protocol to exercise paths that ordinary use may rarely trigger. For a software component, the team might supply a malformed message, a delayed timestamp, or a missing field in an isolated test environment. Before running the test, it records the expected response and the point at which a timeout should be reported. A test that merely produces an error message is not enough if the downstream component continues to use the invalid data as though it were valid.

A test result is strongest when an independent observer can compare actual behavior with a pre-defined expected result. For example, a planner asked to find a route in a graph with no

connection to the goal should report no path after exhausting the reachable graph; if it reaches a time limit first, the result should instead indicate that search was interrupted. The test should check the returned status, not just whether an output file was created. Distinguishing these outcomes protects the next component from treating an incomplete computation as proof of impossibility.

A failure discovered during testing deserves a record even when it is corrected immediately. The record can identify the triggering conditions, the observed and expected behavior, the affected version, the analysis of the cause, the change made, and the results of a repeat test. Regression tests help prevent the same defect from returning in a later version. They should accompany, not replace, new tests for the effects of the change on other parts of the system.

A test plan should also address the selection of reference cases. If experts label the same image differently, resolve the disagreement using the documented procedure or retain an uncertainty label; do not silently choose the label that makes the model look better. When the appropriate answer depends on information absent from the test input, the task definition may need revision. A model cannot reliably infer a fact that the input does not contain merely because an evaluator knows it.

A small evaluation, with uncertainty made visible

Suppose a team tests a binary classifier on 100 independently sampled cases representative of a narrowly defined use. It obtains 90 correct predictions and 10 incorrect predictions. The observed accuracy is 0.90. Under the additional assumptions that cases are independent and share the same probability of correctness, a 95% Wilson confidence interval for that probability is approximately 0.826 to 0.945. This interval expresses sampling uncertainty under the model; it does not include uncertainty from an incorrect reference label, dataset shift, or a test procedure that did not represent deployment.

The interval calculation is not a safety guarantee. Even under the stated assumptions, it concerns the probability of the defined

correctness event in the sampled population. If all ten errors are failures to detect a hazardous condition, the overall accuracy does not describe the consequences adequately. Report a confusion matrix or other error breakdown, the prevalence of relevant conditions, and the sampling design alongside the interval. If cases are clustered by machine or site, an analysis that assumes independent cases can understate uncertainty; the sampling and analysis should reflect that structure.

A related question is what can be inferred when no failures are observed. If n independent trials have the same failure probability p, the probability of observing no failures is $(1 - p)^n$. For n = 100 and a proposed p = 0.03, this probability is 0.97^{100}, approximately 0.0476. Thus observing no failures in 100 trials would be unusual if the true probability were 3% under this simple independent-trial model, but it would not prove the probability is zero. Real robot trials may share conditions and may not satisfy the model's independence or constant-probability assumptions.

The selection of tests is at least as important as their number. A thousand repetitions on one empty corridor provide little evidence about a crowded corridor with a different floor surface. A broad test plan should identify important variations, document which were included, and disclose those not tested. For rare but severe hazards, an evaluation strategy may require engineering analysis and additional forms of evidence rather than relying on a small number of ordinary trials.

An exercise in comparing two policies

A simulation presents a cart with 50 delivery requests. Policy A completes 45, requests assistance on 4, and times out on 1. Policy B completes 47, requests assistance on 1, and times out on 2. These invented results describe only this particular simulation. Policy B completes two more requests, but it also has one more timeout. The totals add to 50 for each policy because the outcomes are defined to be mutually exclusive in this example.

Before choosing a policy, ask what an assistance request and a timeout mean. Assistance may be a desirable response to uncertainty, while a timeout may leave a task unresolved. Compare the policies on the same scenarios when possible and record which cases differ. If B completes a request only because it bypasses a check required by the operating procedure, the completion count is not sufficient evidence that B is preferable. If the difference is caused by random simulation outcomes, repeat the evaluation under a pre-specified protocol and report variability.

The two policies may also place different demands on staff. A policy that frequently requests help could slow a busy workplace; a policy that rarely requests help could conceal unresolved uncertainty. Include the time and information needed for an operator to respond, not just the number of requests. Evaluate whether assistance is available throughout the proposed operating period and what happens when no one acknowledges a request.

For a more detailed analysis, construct a table with one row per scenario and columns for scenario conditions, each policy's outcome, completion time, assistance request, and any observed violation. Keep policy versions and scenario definitions fixed during the comparison. If a policy is changed in response to the results, record the change and evaluate the revised policy separately. This process makes the evidence interpretable and prevents an iterative development result from being presented as an untouched final test.

10. Applications and case studies

Applications and limits

AI can assist with medical image review, translate text, help sort goods, and estimate equipment faults. The same model may perform differently across hospitals, languages, cameras, or time periods. A deployment decision should identify whose errors matter, how a person can challenge or override an output, and how the system will be monitored after release. Human oversight is meaningful only when the operator has sufficient information, authority, and time to act.

A language model may produce a fluent but false explanation. A vision model may be confident about an unfamiliar object. Neither fluency nor confidence alone establishes correctness. The responsible approach is to specify the claim, test against relevant evidence, record failures, and provide a safe response when uncertainty is too high.

Case study: monitoring a crop without making an automatic diagnosis

The following case is hypothetical. A cooperative wants to prioritize manual inspection of crop photographs. Its staff report that inspecting every plant is impractical, but a false assurance that a damaged plant is healthy could delay treatment. The proposed system will flag images for human review; it will not select or apply a treatment. This boundary limits the task, although it does not eliminate the possibility of harm from missed flags or excessive workload.

The first design decision is the unit of analysis. A photograph may contain several leaves, while the operational question concerns a plant. A model that classifies each image needs a rule for combining several images into a plant-level decision. If any image is flagged, the plant might be sent for review, but that rule can increase the number of false flags. If the images cover only one side of a plant, a clear prediction cannot establish that the entire plant is

healthy. The data collection protocol should record coverage as well as the image label.

A team creates a written labeling guide with examples of the categories and a category for cases that cannot be judged from an image. It asks qualified reviewers to label a sample independently and records disagreements. A reviewer may identify a symptom without being able to identify its cause. The label should preserve that distinction: “visible discoloration” is not automatically “disease caused by organism X.” Disputed cases can be reviewed using additional evidence, but a final label should retain uncertainty where the evidence remains inadequate.

The team divides the available material by farm and collection period before model development. A split by image alone could put photographs of the same plant in both training and testing sets. Such a split may reward recognition of the background or plant identity rather than the intended generalization to new plants. If the intended deployment includes a new region, the evaluation should include evidence from that region or state clearly that such evidence is absent. A model evaluated only on the original farms cannot be claimed to work everywhere.

Suppose the held-out evaluation has 100 plants with an independently established visible symptom and 900 without it. A particular threshold flags 80 of the 100 symptomatic plants and 90 of the 900 others. Symptom recall is $80/100 = 0.80$. Flag precision is $80/(80 + 90) \approx 0.471$. The review queue contains 170 plants, and 20 symptomatic plants are not flagged. These hypothetical numbers are not a clinical or agricultural recommendation; they show that a review system can impose substantial work while still missing cases.

The cooperative must decide whether that workload and missed-case rate are acceptable for the proposed use. It may lower the threshold to flag more plants, collect additional views, or choose a different process. Threshold selection belongs on development data; a final test estimates the selected procedure on

separate cases. Reviewers also need a way to report an incorrect label or a new symptom, and the team needs a procedure for investigating those reports. A model's confident output should not prevent a person from escalating an independently observed problem.

This example illustrates a recurring system boundary. The model processes an image; the operational service includes data collection, image quality, model prediction, aggregation, human review, and follow-up. A failure can occur at any of these stages. Reporting a classifier score alone does not establish the quality of the service as a whole.

Case study: a map, a route, and a closed door

A second hypothetical system transports sealed materials through a building. Its planner uses a graph whose nodes are locations and whose directed edges are permitted transitions. A facilities team supplies the map. A navigation component estimates the cart's current location, and a route follower attempts the next transition. These are different responsibilities: the planner can return the least-cost path in its graph even when the map is stale or the localization estimate is wrong.

During a test, the planner selects A → B → C. The cart arrives at the door between A and B and receives a closed-door observation. The stored graph still labels the edge traversable. The plan monitor records the disagreement and prevents the route follower from treating the old map entry as confirmation that the door is open. Depending on an approved operating procedure, it may request help or replan after receiving an updated map. If it cannot determine its own location or the status of the door, the uncertainty must be represented in the response, not hidden behind the last successful plan.

An investigation should record the map version, localization estimate, sensor health, command history, and timestamps. If a worker reports that the door had been scheduled for closure, the question becomes why that information did not reach the map

update process. Replacing the search algorithm might not address the cause. If the door was open but the sensor falsely reported it closed, sensor performance and recovery behavior deserve attention. A trace linking an observed failure to an action taken by the system helps distinguish these cases.

A pre-deployment test matrix can include a normally open door, an expected closure, an unexpected closure, a sensor that reports an invalid value, and an update that arrives late. Each test should identify the setup, expected system behavior, observations to collect, and pass or fail criteria. Tests should also specify what is not being demonstrated. A simulated invalid sensor test can verify a software response to an injected message; it does not by itself verify wiring, mechanical stopping distance, or behavior under all hardware failures.

Comparing approaches to a single task

Consider an automated sorting assistant that proposes one of three destinations for a returned item. A fixed rule might use a product code; a search procedure might choose a route to the selected destination; a classifier might estimate a category from an image when the code is missing; and a human operator might resolve ambiguous cases. These are not necessarily competing solutions. Each addresses a different part of the workflow.

A fixed rule is attractive when the input is reliable, the categories are well defined, and the rule can be checked. It is brittle if codes are absent or wrong. A learned image classifier may handle a wider range of appearances, but it requires representative labeled examples and can fail under changed packaging or illumination. A human can use context that the system lacks, but human review has costs and can be inconsistent or overloaded. A combined design needs to define when each component is used and how disagreements are resolved.

The comparison should include maintenance. A rule base needs a process for updating codes and destinations. A learned model needs monitoring, data governance, and a process for retraining and

re-evaluation. A search graph needs current connectivity and access restrictions. The simplest initial implementation is not necessarily the least expensive over its whole lifecycle, but complexity should be justified by evidence rather than adopted for its own sake.

An evaluation can compare the combined workflow with the existing process. Measure not only category agreement but also the time required, the rate of unresolved cases, the effect of incorrect proposals on operators, and the consequences of missed or delayed items. If operators tend to accept the system's suggestion without checking it, high apparent agreement between human and model may not be independent confirmation. Study the interaction rather than assuming that a human review step always corrects machine errors.

11. Reliability and failure analysis

Handling missing, conflicting, and delayed information

A robust information pipeline distinguishes a valid zero from a missing value. A range reading of zero may have a defined physical meaning, while a missing reading means no usable measurement was received. Converting both to the same numeric value can cause a downstream component to misinterpret the situation. The interface should define valid ranges, units, timestamps, and an explicit representation for invalid or unavailable data.

Consider two reports about a doorway. A building map says the door is open, while a local sensor says it is closed. The reports may refer to different times, different doors, or different definitions of “open.” Before fusing them, check identity, time, and meaning. If the local sensor is current and trustworthy, it may justify revising a stale map entry. If either report is uncertain or the timestamps cannot be compared, the system should preserve that uncertainty and follow its documented response. Taking the majority of two inconsistent reports cannot resolve the disagreement without an additional rule or evidence.

Delay is a property of the entire pipeline. An image can be captured, transmitted, processed, and used at different times. If an object moves during this interval, the image may describe an earlier scene rather than the current one. A system should record acquisition time as well as processing time and assess whether the age of the information is acceptable for the intended decision. A short average delay does not rule out occasional long delays; timeouts and stale-data handling need their own tests.

Data quality checks should be designed around the task. A constant sensor output may indicate a fault, but it can also reflect a genuinely stable condition. A sudden change may be a real event or a measurement error. A check can flag a suspicious pattern for further investigation; it should not automatically erase inconvenient observations without a documented justification. Preserve enough information, subject to privacy and security requirements, to understand why data was rejected or modified.

Failure analysis: tracing a system-level error

Suppose a delivery cart pauses unexpectedly in a corridor. The first observation is that the cart stopped; the cause is not yet known. Possible explanations include a valid obstacle detection, a sensor fault, a stale map, a communication interruption, a control fault, or an intentional stop request. A disciplined investigation keeps these alternatives separate until evidence supports or rules them out.

Begin with a timeline. Record what the sensors reported, which software version was running, which state the controller believed it was in, what command was issued, and whether a person intervened. Timestamps should use a consistent reference or include enough information to interpret clock differences. Missing records are themselves a finding: a log that omits sensor health may prevent the team from distinguishing an actual obstacle from a sensor failure.

Next, compare the observations with the system's expected behavior. If a stop was correctly triggered by a person in the cart's path, the system may have worked as intended even though the

delivery was delayed. If a stale message was treated as a current measurement, the issue may lie in the interface contract between components. A local fix should be checked against related scenarios: increasing a timeout might reduce unnecessary pauses but could also allow the system to act on old information for longer.

A corrective action should be specific and verifiable. “Improve the AI” is not an actionable finding. “Reject range messages older than the defined freshness limit and enter the documented response state” is a more precise candidate, assuming the freshness limit and response are justified for the system. The change needs tests for normal, delayed, missing, and out-of-order messages. A review should also ask whether the same failure mode exists in other sensor interfaces.

Failure analysis should not become a search for a single person to blame. Human actions occur within procedures, interfaces, incentives, training, and resource limits. If an operator was expected to notice a subtle warning among many routine alerts, the interface and workload may be part of the problem. Accountability means establishing who can make and review decisions, while learning from how the entire system behaved.

12. Responsible deployment

Responsible deployment and human authority

Accountability starts before the first model is trained. Identify who defines the task, who supplies and labels data, who approves deployment, who monitors performance, and who can stop the system. These responsibilities need not belong to one person, but gaps and handoffs should be explicit. An operator cannot meaningfully supervise an autonomous system without information about its current state, limits, and available controls.

Privacy requires attention to the whole data lifecycle. A camera used for navigation may also capture people, screens, and documents. The design should consider whether all captured material is necessary, how long it is retained, who can access it,

and how deletion or correction requests are handled under applicable rules. Removing names does not necessarily make data anonymous; images, locations, and timestamps can still identify people. Technical safeguards should be paired with governance and an assessment of the local legal context.

Fairness depends on the application. A speech interface that performs poorly for a group of users can exclude them even if its overall accuracy appears high. A meaningful assessment requires appropriate consent or other lawful grounds for collecting relevant data, careful subgroup definitions, enough examples for useful estimates, and engagement with affected users. One metric cannot capture every form of unequal treatment. When important groups are missing from the evaluation, the limitation should be disclosed and addressed rather than concealed by a single aggregate score.

Security and reliability also interact. An altered map, an unauthorized software update, or a forged sensor message can affect a robot's decisions. Access controls, update integrity, logging, and recovery procedures complement the evaluation of ordinary prediction errors. Their exact design depends on the threat model and system architecture; adding a security feature to a diagram is not evidence that the finished system is secure.

A deployment plan should include a change-control procedure. A new camera, a software library update, a revised map, or a retrained model can alter system behavior even if the user-facing task remains the same. Classify changes by their possible impact, rerun relevant tests, and approve the change through the appropriate process. Preserve version information for the model, calibration, configuration, and software so that an investigation can reconstruct which system produced a particular result.

Human intervention can also introduce risk. An override control placed in an inaccessible location, or a procedure that requires an operator to enter the robot's travel path, may be ineffective or unsafe. Evaluate the whole interaction, including workload, visibility, response time, and training. A nominal “human in the

loop” does not solve a problem if the person is routinely asked to approve decisions too quickly to understand them.

13. Integrated engineering exercise

An integrated design exercise

A library proposes an internal mobile cart that carries books between a return desk and a sorting room. The following design exercise is fictional and does not authorize operating a robot in a library. Its purpose is to connect AI concepts to a real engineering workflow while making assumptions visible.

Step 1: define the service. The cart transports loaded containers on a limited set of routes during staffed hours. A human loads and unloads the containers. The project team records the building areas included in the trial, access rules, maximum intended load, and a procedure for manual recovery. People using the space, including people with mobility or sensory disabilities, are consulted before the requirements are fixed. The team does not assume that all users can hear a warning or move aside.

Step 2: describe the information and actions. The cart has a map, a localization estimate, and sensor observations. Its actions include requesting a route, commanding motion through an appropriate controller, pausing, and asking for assistance. A map records expected layout; it does not establish that a particular corridor is clear now. A localization estimate has uncertainty; it does not prove the cart's exact position. For each input, the design records an owner, update frequency, failure indications, and the system response to invalid or outdated data.

Step 3: separate decision layers. A route planner proposes a path through permitted graph edges. A lower-level motion controller follows an approved segment subject to physical and control constraints. A monitoring function checks conditions defined by the safety design. The distinction matters for diagnosis: if the route is reasonable but the cart overshoots a turn, the problem may lie in estimation or motion control rather than in

graph search. If a route enters a newly restricted corridor, the problem may involve map governance or constraint enforcement.

Step 4: construct evidence. The search component is checked on small graphs with known answers, including unreachable goals and alternative costs. The system is then evaluated in scenarios reflecting the intended environment. Scenarios should cover changes in lighting, temporary obstructions, an invalid sensor reading, a missing map update, and operator intervention. Simulation can exercise logic repeatedly, but the results must be distinguished from physical tests. A qualified team defines and validates physical safeguards and testing procedures before any interaction with the public.

Step 5: decide what happens at the boundary. If an intended corridor is closed, the system should not infer that the graph remains correct merely because a path was computed earlier. A documented procedure might permit replanning after an authorized map update or require assistance. The team specifies who may update permissions and how the cart recognizes the update. A failure to locate the cart or establish the state of the corridor must be handled explicitly rather than assigned an invented confident value.

Step 6: review the result with affected people. Record successful deliveries, interruption causes, time taken, assistance requests, and user concerns. Do not collect identifiable video merely because it is technically available. Review outcomes across the actual range of users and situations, while respecting privacy and applicable rules. A low average delivery time does not compensate automatically for obstructing a wheelchair route or creating an inaccessible interaction. The decision to continue the trial belongs to accountable people who can inspect the evidence and stop the system.

The exercise shows why an AI component cannot be evaluated in isolation. Search supplies a route under a model; sensing and estimation connect the model to the current building; control

attempts to execute it; people define the task, restrictions, and acceptable outcomes. A defect in any link can change the result.

14. Communicating and reviewing evidence

Communicating evidence and limitations

A useful technical report lets a reader reconstruct what was done. It identifies the system version, task, intended operating conditions, data source, comparison method, metrics, number of trials, and deviations from the plan. It distinguishes measured results from assumptions and recommendations. A statement such as “the planner found a lower-cost path on all 20 test graphs” is narrower and more useful than “the planner is always optimal,” unless a separate proof and its assumptions support that stronger claim.

Graphs and tables should be chosen to answer a question. A table of results by environment can reveal a problem hidden in an overall average. A plot of response time should state units, show how many observations contributed, and describe how missing or timed-out runs were handled. Excluding timeouts from an average can make a slow system appear faster. The appropriate summary depends on the operational consequences; for a time-critical task, the distribution and worst observed delays may matter more than the mean alone.

An explanation is not the same thing as an audit trail. A decision tree can give an understandable route through its rules, but the explanation may still depend on incorrect input. A complex model may require a separate explanation method, which itself can be incomplete or misleading. In either case, preserve the input context, model version, output, and relevant human actions subject to privacy and security requirements. An explanation should help a person contest or investigate a result, not merely make the result look persuasive.

A claim should have a clear scope. “No collisions were observed in these 100 simulation runs” describes the observation; “the robot cannot collide” is a much stronger claim that those runs do not

establish. The report should describe the simulated scenarios, the model version, how scenarios were selected, and whether the system completed each run. It should also say whether physical testing or an independent safety assessment was performed. A reader can then decide which inference the evidence supports.

When work is unfinished, label it accordingly. A preliminary result can be useful to a team if its status and known gaps are visible. A publication-ready textbook or safety case requires a different level of source verification, technical review, and editorial checking. Giving a document an authoritative title does not supply the missing evidence. Versioning and a dated record of revisions help readers distinguish a working draft from a later reviewed edition.

A professional checklist for an AI proposal

Before development begins, write down the task in terms of the people affected, the intended benefit, and the relevant environment. Identify what the system will not do. Ask whether the proposed inputs can be collected lawfully and whether a simpler process already meets the need. Make a list of decisions that will remain with people and identify the person or organization responsible for each one.

Before evaluating the system, specify the outcome measures, comparison baseline, dataset or scenario selection, and treatment of failures and timeouts. Check for leakage between development and evaluation data. Plan how to examine performance across relevant operating conditions, and record where evidence is insufficient. Confirm that the reference labels or measurements are themselves appropriate for the claim.

Before deployment, review the operating domain, the response to missing or conflicting information, access controls, update procedures, and the route for complaints or corrections. Assign monitoring and maintenance responsibilities. Define conditions under which the system will be paused or withdrawn. These questions are not a complete safety or legal assessment, but they make omissions visible and establish a basis for specialist review.

15. Summary and review

Chapter synthesis

AI offers several ways to turn information into action: searching a state space, applying explicit rules, updating beliefs, learning patterns, and selecting actions toward goals. These methods solve different problems and rely on different assumptions. Search needs a state and transition model; logical reasoning needs meaningful premises; statistical methods need relevant data and evaluation; a physical agent needs reliable sensing and actuation. Good engineering connects these components to an explicit task and examines what happens when each assumption fails.

Review and investigation

1. Describe the observations, actions, environment, and performance measures for an automated crop-inspection robot. Which important variables are hidden?

2. Construct a weighted graph where the route with the fewest edges is not the least-cost route. Explain why breadth-first and uniform-cost search may return different paths.

3. A heuristic returns zero for every node. What does A* prioritize? What information has the heuristic failed to provide?

4. Propose a way to detect a harmful change in performance after deployment without collecting unnecessary personal information.

5. On a four-neighbor unit-cost grid, compute the Manhattan heuristic from (2, 1) to (5, 5). Explain why the value need not remain admissible if diagonal moves are allowed at unit cost.

6. Apply Bayes' rule when a fault occurs in 10% of machines, the alarm detects 90% of faults, and the false-positive rate is 5%. Show the denominator as well as the result.

7. Describe two tests that distinguish an incorrect search implementation from an inaccurate map of the environment.

8. In the three-room constraint problem, explain why three mutually adjacent rooms cannot use only two channels if neighboring rooms must differ.

9. A delivery planner has not returned an answer before its time limit. Explain why this differs from proving that no route exists.

10. Identify an assumption in the discounted-return example and explain one situation in which it would be inappropriate to optimize that number alone.

Selected guidance: For question 5, the value is $|5 - 2| + |5 - 1| = 7$. For question 6, the denominator is $0.90(0.10) + 0.05(0.90) = 0.135$ and the posterior is $0.09/0.135$, approximately 0.667. For question 8, after two adjacent rooms receive different channels, the third is adjacent to both and has no remaining channel. The remaining questions require a reasoned explanation rather than a unique numerical answer.

2. Robotics Fundamentals

A robot is an integrated physical system

A robot is commonly understood as a programmable machine that senses or otherwise obtains information and acts on the physical world. Its capabilities arise from the interaction of structure, actuators, sensors, computation, power, and control. A robotic arm attached to a workbench has a different workspace and risk profile from a mobile robot in a corridor. Both can use AI, but neither requires a learning model in every subsystem.

Industrial manipulators, mobile platforms, aerial vehicles, surgical devices, and agricultural robots are designed around different tasks. Comparisons require precise requirements: payload, reach, speed, accuracy, repeatability, operating environment, and acceptable risk. Accuracy describes closeness to a desired value; repeatability describes how consistently a system returns to the same result. A robot can be repeatable yet consistently miss the target because of calibration error.

Links, joints, and motion

A manipulator can be described as links connected by joints. A revolute joint rotates; a prismatic joint translates. A degree of freedom is an independent coordinate needed to describe a permitted motion. Count carefully: the number of motors need not always equal the number of independent degrees of freedom, because mechanisms can be coupled.

Consider a planar arm with two rigid links of lengths L1 and L2. Let the first joint angle be θ1, measured from the horizontal, and let the second angle θ2 be relative to the first link. The tool position is $x = L1 \cos(\theta1) + L2 \cos(\theta1 + \theta2)$, $y = L1 \sin(\theta1) + L2 \sin(\theta1 + \theta2)$. Here x and y are coordinates in the base frame. For L1 = 1 m, L2 = 0.5 m, θ1 = 0°, and θ2 = 90°, the tool is at (1, 0.5) m. This is a forward-kinematics calculation: joint positions are known and tool

position is found. Real mechanisms also have backlash, flex, joint limits, and calibration errors that this simple model omits.

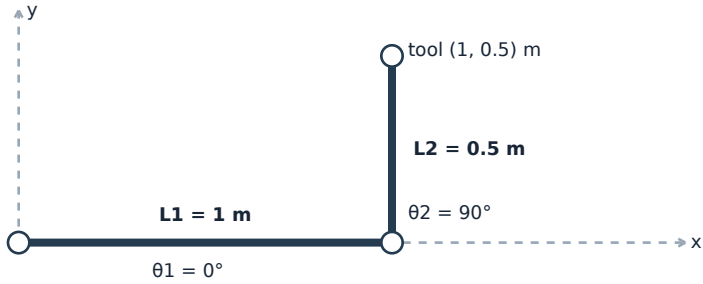


Figure 2.1. Ideal planar two-link arm at θ1 = 0 degrees and relative θ2 = 90 degrees. The diagram is illustrative, not an engineering drawing.

Actuators, transmissions, and power

Electric motors convert electrical energy into mechanical motion. A DC motor, a stepper motor, and a servo system are not interchangeable labels: a servo system generally includes a motor, feedback, and a controller. Gears can trade output speed for torque, but introduce losses, inertia, and possibly backlash. Pneumatic actuators offer a different combination of speed, compliance, and infrastructure requirements; hydraulic systems can provide high forces but bring leakage and maintenance concerns.

Torque is the turning effect of a force. In the simple perpendicular case, $\tau = F\,r$, where τ is torque in newton-metres, F is force in newtons, and r is moment arm in metres. A 10 N load at 0.2 m produces 2 N·m of static torque about the joint. This is not a complete motor specification: acceleration, link mass, friction, gearbox efficiency, duty cycle, orientation, and a justified safety margin must also be considered.

Sensors and uncertainty

An encoder measures movement of a shaft or joint, while a camera or range sensor provides information about the outside world. An inertial measurement unit senses angular velocity and acceleration; estimating position from these readings alone can accumulate substantial drift. Sensors have finite resolution and can be biased, noisy, delayed, or out of range. Calibration relates measurements to physical quantities and must be checked under appropriate conditions.

Suppose a distance sensor reports 0.95 m when a reference measurement is 1.00 m. The signed error is -0.05 m. A single measurement does not establish a systematic bias: take repeated readings at multiple distances, report the reference method and uncertainty, and examine whether the discrepancy varies with range or surface material.

Architecture and safety

A common architectural description is sense, estimate, plan, and act, repeated as new measurements arrive. This is a conceptual decomposition rather than a promise that every robot runs a single sequential loop. A fast low-level motor controller can operate alongside a slower planner. Communication delay and missed deadlines can cause a correct decision to arrive too late.

Safety starts with identifying hazards: crushing, unexpected motion, electrical energy, dropped loads, stored pneumatic pressure, and interactions with people. Reducing risk may require guarding, safe separation, power isolation, suitable safety-rated components, and validated procedures. An emergency stop is a complementary protective measure, not a substitute for a risk assessment or all other protective measures. Consult applicable regulations and relevant standards for the specific machine and jurisdiction.

From task to machine: a design example

Imagine a team asked to move small sealed containers from a loading station to a nearby inspection station. The task description does not yet specify a robot. The team first observes how containers are loaded, how much they weigh, where people stand, what happens when an item is damaged, and who has authority to stop work. It records dimensions, loads, cycle times, workspace restrictions, cleaning requirements, and environmental conditions. A fixed arm, a mobile cart, or a manual conveyor might each be considered; choosing a robot before understanding the task risks solving the wrong problem.

The requirements should distinguish the heaviest expected payload from the mass of the end effector that holds it. A payload rating in a manufacturer's specification must be interpreted using that manufacturer's conditions, including load position, motion profile, and mounting. An arm nominally rated for a given mass may not be suitable if the load is held farther from the mounting flange than permitted. Reach, moment, acceleration, and the mass of cables and fixtures can all matter.

The team draws a system boundary. The robot includes mechanical parts, drives, sensors, a controller, and a power system. The surrounding system also includes fixtures, guards, work surfaces, people, and procedures. A change to a fixture can create a pinch point even if the robot itself has not changed. Assessing an integrated installation therefore differs from checking a component in isolation.

A useful specification says how a requirement will be tested. “Place containers accurately” could be expressed as an allowed placement error under a stated load, at stated positions, speeds, and environmental conditions. The method of measurement and its uncertainty must be adequate for the tolerance being checked. A single successful placement is not evidence of repeatability across a range of conditions.

Robot families and their trade-offs

A fixed-base manipulator uses a series or arrangement of joints to move an end effector through a workspace. It can be useful where work is brought to a known station, but the base does not move to reach work elsewhere. A mobile robot carries its base through an environment; this expands its working area but introduces localization, floor-contact, obstacle, and battery issues. A mobile manipulator combines a base and an arm, and must account for how arm motion and load affect the stability of the entire machine.

A serial manipulator has links connected in a chain from base to tool. It often offers a flexible workspace, while errors and deflections in earlier joints can affect the position of later links. A parallel mechanism uses multiple kinematic chains to connect a base and moving platform. Such mechanisms can be stiff and fast for suitable tasks, but their workspace and singularities depend strongly on their geometry. Neither arrangement is universally superior; the task, workspace, load, and cost guide the selection.

Legged, wheeled, aerial, and underwater machines interact with their environments in different ways. Wheels can be efficient on suitable surfaces but may struggle with steps or soft ground. Legs can negotiate some uneven terrain, at the price of mechanical and control complexity. Aerial vehicles must manage lift, energy, weather, and the consequences of loss of flight. These are general trade-offs, not assurances about a particular design. Specialized systems require domain-specific engineering and safety review.

Frames, coordinates, and units

A position is meaningful only relative to a reference frame. Suppose a point has coordinates (1.2, 0.4) m in a frame fixed to a robot's base. This says the point is 1.2 m along the base frame's x-axis and 0.4 m along its y-axis, using the stated axis directions. It does not by itself say where the point lies in a room. To express the point in the room frame, the position and orientation of the base frame relative to the room must be known.

For a simple translation with no rotation between frames, if the base origin is at (2, 1) m in the room frame, the point at (1.2, 0.4) m in the base frame is at (3.2, 1.4) m in the room frame. The addition is valid because the axes are assumed parallel and have the same scale. If the base is rotated, coordinates must be rotated before translation; adding the pairs directly would be incorrect.

For a planar rotation by angle θ, $x' = x \cos \theta - y \sin \theta$ and $y' = x \sin \theta + y \cos \theta$. Here (x, y) are coordinates in the original frame and (x′, y′) are coordinates after applying the stated rotation to the vector. A 90-degree counterclockwise rotation maps (1, 0) to (0, 1). A complete frame transformation must also state whether it maps coordinates from the robot frame to the room frame or in the reverse direction. Frame names and direction are not optional decorations: confusing them can reverse a motion command.

Radians and degrees are both used to express angles, but equations and software must use a consistent convention. One complete turn is 360 degrees or 2π radians; 90 degrees is π/2 radians. Most Python trigonometric functions take radians. Passing 90 directly to such a function does not mean 90 degrees. A unit conversion error can be large even though the program executes without an exception.

Workspace and geometry of a simple arm

For an ideal two-link planar arm with lengths L1 and L2, the distance from the base to the end point cannot exceed L1 + L2. It also cannot be less than $|L1 - L2|$ when both joints can rotate without limits and the links are allowed to fold freely in the mathematical model. With L1 = 1 m and L2 = 0.5 m, the ideal reachable distances lie between 0.5 m and 1.5 m. Actual joint limits, collisions, cable routing, tool shape, and required orientation can make the usable workspace smaller.

Forward kinematics answers where the tool is for specified joint angles. Inverse kinematics asks which joint angles give a desired tool pose. Inverse kinematics may have more than one solution, one

solution, or no solution. For example, an elbow-up and an elbow-down configuration may reach the same position in a planar model. The two configurations need not be equally suitable: one may collide with a fixture or violate a joint limit. Position alone also does not always determine the desired tool orientation.

The earlier example used L1 = 1 m, L2 = 0.5 m, θ1 = 0 degrees, and θ2 = 90 degrees, producing (1, 0.5) m. If instead θ1 = 90 degrees and θ2 = 0 degrees, both links point upward and the end point is (0, 1.5) m, apart from numerical rounding. If both angles are zero, the arm is fully extended along the positive x-axis at (1.5, 0) m. These examples assume the second angle is measured relative to the first link; a different convention changes the equations.

A singular configuration is one in which the local relationship between joint velocities and tool velocity loses rank. For the ideal planar two-link arm, full extension is one such configuration. Near it, producing certain small tool motions may require large joint speeds or may not be possible in the desired direction to first order. A singularity is a property of a specified model and configuration, not a physical wall. Detailed Jacobian analysis belongs to Chapter 24, but the design implication is already important: a geometrically reachable point may still be difficult to approach or control.

Motion, force, and energy

Position, velocity, and acceleration describe different aspects of motion. Velocity is the rate of change of position; acceleration is the rate of change of velocity. A command to move a tool 0.2 m says nothing by itself about how quickly it should move or how it should start and stop. A trajectory adds timing to a path. Abrupt changes in the requested motion can require large forces, excite vibration, or exceed the capabilities of a drive.

For a simple constant-speed example, travelling 0.6 m in 3 s gives a speed of 0.2 m/s. This calculation assumes constant speed over the entire interval. A real robot starting from rest must accelerate and later decelerate, so its peak speed would be higher than the

average if it completed the same displacement in the same time. A design should account for the actual trajectory rather than sizing components from average speed alone.

Mechanical power at a rotating shaft is $P = \tau\omega$. Here P is power in watts, τ is torque in newton-metres, and ω is angular velocity in radians per second. At 0.1 N·m and 100 rad/s, the mechanical power is 10 W. If the electrical input at the same operating point is $12\ \text{V} \times 2\ \text{A} = 24\ \text{W}$, the illustrative efficiency is $10/24 \approx 41.7\%$. This calculation assumes the measurements refer to the same steady operating condition and that the stated electrical voltage and current represent input power appropriately. Losses become heat or other forms of energy; efficiency changes with operating point.

Energy limits matter for mobile systems. A battery's nominal voltage and capacity can provide a rough energy estimate, but actual available energy depends on current, temperature, aging, cutoff settings, and other factors. A robot also needs reserve for startup, communication, safe stopping, and recovery. A spreadsheet estimate is a starting point for measurement, not a guarantee of runtime or a justification for bypassing battery protection.

Selecting motors and transmissions

Motor selection starts with the required motion and load, not the motor's largest advertised number. A useful calculation considers continuous and peak torque, speed, acceleration, operating time, thermal conditions, gearbox losses, and the mechanism's inertia. Peak torque may be available only briefly; selecting a motor on peak torque alone can overheat it during sustained work. The motor, drive, transmission, and power supply must be compatible as a system.

For a simplified static arm held horizontal, a 1 kg point payload at a 0.3 m moment arm creates approximately $1 \times 9.81 \times 0.3 = 2.943$ N·m about the joint due to gravity. This ignores the arm's own mass, friction, acceleration, and the gripper. If a gearbox has a reduction ratio of 10:1, defined here as ten motor revolutions per

one output revolution, and an assumed efficiency of 0.8 at that operating point, the approximate motor torque needed for this load is 2.943/(10 × 0.8) ≈ 0.368 N·m. The result is an illustrative lower-level calculation, not a complete design rating.

The corresponding ideal speed relationship is output speed = motor speed/10 for this reduction. Torque and speed are traded through the transmission, while losses and thermal limits remain. A manufacturer may define efficiency or gear ratio differently, so use the convention and test conditions in the relevant documentation. Include uncertainty and a justified margin in an actual design, then verify against the complete operating cycle.

Backlash is lost motion associated with clearance between transmission components. When a joint reverses direction, the motor may rotate slightly before the output takes up the clearance. It can degrade position accuracy and complicate control. Compliance is elastic deformation under load; even without backlash, a loaded arm can deflect. These effects are not necessarily constant, so a correction fitted at one load or direction may not work everywhere.

Wheels and basic mobile motion

For an ideal differential-drive platform, two driven wheels are placed on a common axle. Let the left and right wheel ground speeds be vL and vR, and let b be the distance between their contact lines. Under the ideal rolling model without lateral slip, the speed of the midpoint is v = (vR + vL)/2 and the yaw rate is ω = (vR − vL)/b. Positive yaw is defined here as turning left when the right wheel moves faster than the left. Wheel speeds are in metres per second, b is in metres, and ω is in radians per second.

If vL = 0.2 m/s, vR = 0.4 m/s, and b = 0.5 m, then v = 0.3 m/s and ω = 0.4 rad/s. The midpoint follows a curved path in the ideal model. If both wheel speeds are 0.3 m/s, ω = 0 and the ideal midpoint travels straight. If the wheel speeds are equal and opposite, the ideal midpoint has zero forward speed and the chassis rotates in

place. Actual behavior differs when tires slip, effective wheel radii differ, or the ground is uneven.

Wheel encoders can estimate travel by accumulating wheel rotation. With an effective wheel radius r and angular displacement $\Delta\varphi$ in radians, the ideal distance is $\Delta s = r\Delta\varphi$. For $r = 0.05$ m and $\Delta\varphi = 2\pi$ radians, one revolution corresponds to about 0.314 m of travel without slip. Encoder resolution and wheel diameter measurements influence the estimate. Even a carefully calibrated odometry system accumulates error when its assumptions are violated, so it should not be treated as a perfect measurement of position.

Sensors: selection and interpretation

A proprioceptive sensor measures a quantity associated with the robot itself, such as joint position, wheel rotation, or motor current. An exteroceptive sensor measures something about the environment, such as reflected light or distance to a nearby surface. The distinction depends on the use: a force sensor at a gripper can reveal both load on the robot and interaction with an object. Classifying a device is less important than specifying what it measures, where, when, and with what limitations.

A sensor's resolution is the smallest change represented by its output under a specified interpretation. Accuracy concerns closeness to a reference value; precision or repeatability concerns the spread of repeated measurements under stated conditions. Resolution alone does not establish accuracy. A device that reports millimetres may have a bias of several centimetres, and averaging many readings will not automatically remove a constant bias.

For an ideal 10-bit analogue-to-digital converter spanning 0 to 5 V, there are $2^{10} = 1{,}024$ possible output codes. The nominal step size is 5/1,024 V, or approximately 4.88 mV per code, under an idealized uniform quantization convention. This is not the same as guaranteed measurement accuracy: reference-voltage error, noise, sensor error, and converter nonlinearity also matter. A real

converter's data sheet defines its transfer characteristics and operating limits.

Timing is another sensor property. A measurement has an acquisition time and may reach the controller later. A camera frame processed 200 ms after capture describes an earlier scene; a moving object may no longer be at the measured position. Filtering can reduce some noise but may add delay. The relevant engineering question is whether the measurement, together with its uncertainty and age, is suitable for the decision being made.

Calibration and measurement practice

Calibration relates an instrument's indications to reference values under specified conditions. A simple linear calibration can be written $y = ax + b$, where x is a raw sensor reading and y is an estimate in physical units. If a sensor reports $x = 100$ at a reference distance of 0.5 m and $x = 300$ at 1.5 m, the line through these two points has slope $a = (1.5 - 0.5)/(300 - 100) = 0.005$ m per count and intercept $b = 0$ m. A raw value of 200 then corresponds to 1.0 m under this model.

Two points define a line but do not establish that the sensor is linear across the entire range. Collect additional reference points, repeat measurements, examine residuals, and include the uncertainty of the reference instrument. A calibration performed on one surface or at one temperature may not transfer to another. Record the procedure and conditions, then recheck calibration after relevant changes or according to an appropriate maintenance plan.

If repeated readings at the same reference are 0.98, 1.00, and 1.02 m, their mean is 1.00 m and their range is 0.04 m. The range is a simple description of spread for these three observations, not a complete uncertainty analysis. Three readings are too few to establish performance across an operating range. A measurement plan should distinguish random variability, systematic bias, environmental effects, and the reference instrument's uncertainty.

Feedback, commands, and timing

A motor command describes an intended input; an encoder reading reports a measured quantity. Without feedback, a controller may issue a fixed command and assume the resulting motion occurred. In a closed loop, the controller compares a target with a measurement and adjusts the command. Feedback can reduce some effects of disturbances and model error, but it can also produce oscillation or instability if the control design is poor. Chapter 6 develops these ideas mathematically.

A robot commonly uses several rates. A high-level task planner might update when a task or map changes, while a motion controller operates more frequently. Exact rates depend on the system, plant dynamics, sensors, communication, and safety requirements. It is not enough to say that a loop is “real time”: a requirement should identify the deadline, allowable variation, and response to missed or invalid updates.

A command can fail because the actuator saturates, the power supply cannot deliver the required current, a joint limit is reached, or a sensor disagrees with the expected state. A system should define how these conditions are detected and handled. A software command to stop may not be equivalent to a safety-rated protective stop or to removal of hazardous energy. The actual protective measures and verification belong to the specific installation and its risk assessment.

Simulation laboratory: two-link forward kinematics

The following program computes end-point position for the ideal planar arm described above. It uses metres for link lengths and degrees for the function's input angles, converting them to radians before calling the trigonometric functions. It does not simulate collisions, dynamics, joint limits, or a physical robot.

```
  import math
```

```
def end_point(l1, l2, theta1_deg, theta2_deg):
    if l1 < 0 or l2 < 0:
        raise ValueError("link lengths must be nonnegative")
    t1 = math.radians(theta1_deg)
    t2 = math.radians(theta2_deg)
    x = l1 * math.cos(t1) + l2 * math.cos(t1 + t2)
    y = l1 * math.sin(t1) + l2 * math.sin(t1 + t2)
    return x, y

def near(point, expected):
    return all(abs(a - b) < 1e-9 for a, b in zip(point, expected))

assert near(end_point(1, 0.5, 0, 90), (1, 0.5))
assert near(end_point(1, 0.5, 90, 0), (0, 1.5))
assert near(end_point(1, 0.5, 0, 0), (1.5, 0))
print(end_point(1, 0.5, 0, 90))
```

Save the program as `two_link.py` and run it with Python 3. It should print a point numerically close to `(1.0, 0.5)`; floating-point rounding can affect the displayed digits. The tests use a tolerance rather than exact equality because trigonometric calculations are approximate. Change θ2 to 180 degrees and predict the position before running the code. The model gives (0.5, 0) m, but a real arm may not be able to fold to that configuration.

To investigate conventions, change the code so that the second input angle is measured from the fixed horizontal rather than relative to the first link. The second link's direction would then use that absolute angle directly. Compare results for θ1 = 90 degrees and a second angle of zero: the absolute-angle version points the first link upward and the second link horizontally. Document the convention in the function name or interface. Never send the computed coordinates to physical equipment without a separate validated integration and safety process.

Safety through the system lifecycle

Risk assessment begins with identifying reasonably foreseeable hazards and situations in which people may encounter them. These can include moving machinery, unexpected restart, electrical energy, hot components, dropped objects, sharp edges, and stored mechanical or pneumatic energy. The team considers who can be exposed, the severity and likelihood factors required by the applicable method, and how each risk will be reduced. The resulting measures and their verification must be appropriate to the application and jurisdiction.

Inherently safer design can remove or reduce a hazard, for example by avoiding an exposed pinch point where practical. Guards and protective devices can reduce access to hazards that remain. Information, training, and procedures are important, but they should not be relied upon as substitutes for necessary engineering safeguards. An emergency stop complements other measures; it does not replace guarding, proper design, isolation, or a risk assessment.

Maintenance and recovery deserve particular attention. A robot may retain hazardous energy after a normal stop, and gravity can move an arm or drop a load if power is removed. Isolation and control of hazardous energy require procedures and equipment appropriate to the system. A simulation exercise cannot validate an emergency stop or an energy-isolation procedure. Qualified personnel should follow the manufacturer's instructions, site rules, and applicable standards and regulations.

A change to tooling, speed, software, layout, or operating practice can change the hazards. The team should assess the change, update documentation, and revalidate relevant protective functions before returning equipment to service. Near misses, unexpected motions, and operator reports are opportunities to investigate system behavior; dismissing them because no injury occurred can leave an underlying hazard unresolved.

Integration and engineering evidence

Integration proceeds from interfaces. A mechanical interface specifies mounting, loads, and alignment. An electrical interface specifies voltage, current, connectors, grounding, and protective requirements. A software interface specifies messages, units, timing, valid ranges, and behavior on error. Each interface should state assumptions on both sides, so that one component does not silently depend on behavior the other never promised.

A staged test plan can begin with a model or simulation, then examine individual components in a controlled setting before testing the integrated system. For the two-link calculation, a unit test checks a mathematical function; it does not test a motor, encoder, fixture, or safety function. For a real system, verification must include the physical interfaces and protective measures appropriate to the hazards. The order and controls for physical testing must be set by qualified personnel.

Document both successes and limitations. Record the robot configuration, load, speed, calibration, test environment, and measurement method. If an accuracy measurement is made only at one point with no load, report that condition instead of presenting the result as a workspace-wide specification. When a component is replaced or a payload changes, identify which tests and assessments must be repeated. Good engineering evidence makes later maintenance and investigation possible.

Chapter summary

A robot is a physical system in which mechanics, power, sensing, computation, and control interact. Coordinate frames and units make geometric statements precise; actuator and transmission calculations connect desired motion to physical loads; sensors provide imperfect, time-dependent measurements; and feedback uses those measurements to influence behavior. Models are valuable because they expose assumptions, but a result for an ideal model does not establish performance or safety of a complete installation. Requirements, risk assessment, measured evidence, and lifecycle maintenance connect theory to practice.

Review and investigation

1. Sketch a mobile robot and label power, computation, sensors, actuators, transmission, and an emergency-stop arrangement. Explain what happens when a sensor fails.

2. For the two-link arm, calculate the position when both joint angles are zero. Explain why the answer has the units of length.

3. A gripper repeatedly stops 3 mm to the left of its target. Distinguish repeatability from accuracy and propose a calibration experiment.

4. Compare the actuator requirements of a lightweight educational arm and a continuously operating industrial gripper. Include duty cycle and maintenance.

5. For a wheel with radius 0.1 m, calculate ideal distance travelled in one revolution, assuming rolling without slip.

6. A motor shaft delivers 0.2 N·m at 50 rad/s. Calculate its mechanical output power and state the assumptions.

7. For the differential-drive convention in this chapter, calculate v and ω when vL = 0.1 m/s, vR = 0.3 m/s, and b = 0.4 m. Which direction does the ideal robot turn?

8. Explain why a sensor with a small quantization step can still give biased readings.

9. Distinguish a path from a trajectory. Why can the same geometric path require different torque at different speeds?

10. Prepare a test plan for a nonmoving simulated arm model that distinguishes a frame-conversion error from a wrong link length. State expected results and limitations.

Selected guidance: In question 5, the distance is $2\pi(0.1) \approx 0.628$ m. In question 6, $P = 0.2 \times 50 = 10$ W at the stated operating point. In question 7, $v = 0.2$ m/s and $\omega = 0.5$ rad/s; with the stated sign convention the robot turns left. Questions 8–10 require explanation and a justified procedure.

3. Machine Learning

The learning problem

Machine learning uses data or experience to fit a model for a task. In supervised learning the dataset contains inputs x and associated targets y; the model predicts a target for a new input. A regression model predicts a numerical quantity such as energy consumption. A classifier predicts categories such as acceptable or defective. A model does not automatically discover causal relationships: a useful prediction can arise from a correlation that later changes.

Consider predicting motor current from load. If all training measurements were collected at one temperature, a model may behave poorly at another. Record how examples were sampled, what the labels mean, and whether important operating conditions are represented. Missing data, mislabeled samples, and changes in sensor placement can matter more than the choice between two sophisticated algorithms.

Training, validation, and test sets

The training set fits model parameters. The validation set supports choices such as features, model family, and hyperparameters. The test set estimates performance after those choices are fixed. If repeated measurements from one machine are divided randomly across all three sets, the model may benefit from machine-specific signatures. A split by machine, site, person, or time can better match the deployment question. Any preprocessing that learns from data, including scaling and imputation, must be fitted using the training portion within each split to avoid leakage.

A test estimate has uncertainty. Twenty examples cannot provide the same precision as a large, representative sample. Report the number and nature of examples, the metric, and important subgroup results; do not describe a single score as a universal property of the model.

Linear regression worked by hand

A simple model is $\hat{y} = wx + b$. The symbol x denotes an input, w a slope, b an intercept, and $\hat{y}$ the predicted output. For points (1, 3), (2, 5), and (3, 7), $w = 2$ and $b = 1$ give predictions 3, 5, and 7. Mean squared error is $\mathrm{MSE} = (1/n)\,\Sigma(y_i - \hat{y}_i)^2$, where n is the number of examples, y_i is a measured target, and $\hat{y}_i$ is its prediction. The errors are all zero on these three points, so the training MSE is zero. This does not prove that future predictions will be accurate.

Suppose a different model predicts 2, 5, and 8. Its residuals $y - \hat{y}$ are 1, 0, and -1; the MSE is $(1 + 0 + 1)/3 = 2/3$. Squaring penalizes large errors more strongly and gives the metric squared target units. Root mean squared error restores the units of the target, but can still conceal unequal errors across operating regimes.

Classification and trade-offs

A confusion matrix separates true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN), relative to a stated positive class. Precision = TP/(TP + FP) asks what fraction of positive predictions were correct. Recall = TP/(TP + FN) asks what fraction of actual positives were found. With TP = 40, FP = 10, FN = 20 and TN = 930, precision is 40/50 = 0.80 and recall is 40/60 ≈ 0.667. Accuracy is 970/1000 = 0.97. The apparently high accuracy hides the missed positive cases because negatives are common.

A decision threshold changes the balance between false alarms and missed cases. Select it using a validation set and the consequences of errors; then evaluate the fixed threshold on separate test data. When the relevant denominator is zero, the corresponding metric needs an explicit convention rather than an unqualified numerical claim.

Unsupervised learning and generalization

Clustering groups examples according to a chosen similarity measure. In k-means, an algorithm alternates between assigning points to a nearest centroid and updating centroids to the means of their assigned points. Results depend on scaling, initialization, the number of clusters, and the distance measure. A cluster is not automatically a meaningful real-world category, and the method can converge to a local rather than global optimum.

Overfitting occurs when a model captures details specific to the training set that do not transfer to new cases. Underfitting occurs when it cannot capture a relevant pattern even on the training data. Regularization, simpler models, suitable data, and careful validation can help, but none replaces a representative test of the actual task.

Laboratory and review

Use the following standard-library Python fragment to verify the classification example. The assertions serve as small regression tests; they do not validate a deployed model.

```
tp, fp, fn, tn = 40, 10, 20, 930
precision = tp / (tp + fp)
recall = tp / (tp + fn)
accuracy = (tp + tn) / (tp + fp + fn + tn)
assert abs(precision - 0.8) < 1e-12
assert abs(recall - 2/3) < 1e-12
assert abs(accuracy - 0.97) < 1e-12
print(f"precision={precision:.3f}, recall={recall:.3f}")
print(f"accuracy={accuracy:.3f}")
```

1. Explain why a random row split can exaggerate performance when many rows come from the same robot.

2. Recalculate precision and recall if five false positives become true negatives and all other counts remain unchanged.

3. Plan a dataset for predicting motor faults. Describe consent where appropriate, label verification, a baseline, and a test split that reflects future use.

4. Computer Vision

From light to an array

Computer vision seeks information about a scene from images or video. A camera records a sampled, quantized response to light; the array is not the scene itself. Illumination, exposure, lens distortion, sensor noise, and movement all influence the captured data. A dark object under strong illumination can produce a similar pixel value to a lighter object in shadow. Consequently, an algorithm needs assumptions about the acquisition process as well as the image contents.

A grayscale image can be represented as I(r, c), the intensity at row r and column c. A color image usually has multiple channels. The numeric range depends on the representation: an unsigned 8-bit channel commonly uses integers from 0 to 255. Coordinate conventions vary, so specify whether the first index denotes a row or an x-coordinate and where the origin is located.

Filtering and edges

A spatial filter computes a new value from a neighborhood of pixels. For a simple one-dimensional example, take values 2, 5, and 8 and average them with weights 1/3 each: the output is 5. Smoothing reduces some random variation but can blur fine details and boundaries. A median filter replaces the central value by the median in a neighborhood and can be useful for isolated impulse noise; it does not remove all forms of noise.

An edge detector looks for rapid image-intensity change. In a one-dimensional sequence [10, 10, 30, 30], adjacent differences are [0, 20, 0], showing a transition between the second and third samples. Real image gradients have horizontal and vertical components, and an intensity edge need not be an object boundary: it may be a shadow, reflection, or texture. Threshold choice trades missed edges against spurious detections.

Segmentation, detection, and recognition

Segmentation assigns pixels to regions. A binary threshold may classify a pixel as foreground if its intensity exceeds a chosen value; it works only if the selected intensity rule separates the relevant regions well enough. Semantic segmentation assigns a class to each pixel, whereas instance segmentation distinguishes individual objects of the same class. Detection typically identifies objects with a class and a bounding box; classification assigns a label to an image or region. These are related but different tasks, and their evaluation must match the output.

For a detected box, intersection over union (IoU) is the area of overlap divided by the area of the union of the predicted and reference boxes. If each box has area 100 square pixels and their overlap is 60 square pixels, the union is $100 + 100 - 60 = 140$ and $\text{IoU} = 60/140 \approx 0.429$. A higher IoU indicates better geometric agreement under this measure, but it does not by itself establish that the class label is correct.

Geometry and calibration

In an ideal pinhole camera, a point with camera-frame coordinates (X, Y, Z), with $Z > 0$, projects to $u = f_x X/Z + c_x$ and $v = f_y Y/Z + c_y$. The quantities f_x and f_y are focal lengths expressed in pixel units; c_x and c_y give the principal point in pixels. For $f_x = 500$ pixels, $X = 0.2$ m, $Z = 2$ m, and $c_x = 320$ pixels, $u = 370$ pixels. The ratio X/Z is dimensionless. Real lenses distort images, and calibration estimates parameters from observations; uncertainty and calibration quality matter for measurement.

One image does not generally determine absolute depth without additional assumptions or information. Stereo cameras, known object dimensions, motion, structured light, and other sensors can add constraints. Later robot-vision work connects these estimates to coordinate transforms and action.

Design and evaluation

For an inspection task, begin with controlled illumination and a camera positioned to reveal the defect. Acquire images representing good and defective examples, normal variation, and foreseeable failure conditions. Separate data by production run where appropriate. Evaluate detection on a held-out set with reference labels, then review false negatives and false alarms with domain specialists. A system that works only on one bench is not necessarily ready for an operating factory.

Image datasets may contain identifiable people, locations, or confidential processes. Minimize collection, control access, and establish a suitable legal and ethical basis. Operational monitoring should include image-quality checks, not just the classifier's confidence.

Review and investigation

1. Give an example where a brightness threshold fails because of a shadow. Suggest a change to the acquisition setup.

2. Calculate IoU if two boxes have areas 50 and 70 and overlap by 20 square pixels.

3. In the pinhole model, what happens to $u - c_x$ when Z doubles and all other terms stay fixed?

4. Design a test that measures how motion blur affects defect detection. State the controlled variables and the result you would record.

5. Natural Language Processing

Language as an engineering problem

Natural language processing (NLP) builds systems that process human language. Language is not a simple codebook. A word may have several meanings, speakers omit shared context, and the same request can be phrased in many ways. “Move the box beside the door” can refer to more than one box or door. A robot acting on that sentence should ask for clarification when the ambiguity matters, rather than quietly selecting an arbitrary interpretation.

A corpus is a collection of language examples. Its sources, languages, dialects, permissions, and labeling procedures affect what a model can learn and who it serves. Text may contain names, medical details, or other sensitive information. Data minimization and access controls are part of the design, not an afterthought.

Tokens and representations

Tokenization divides a string into units for processing. Depending on the method, a token can be a word, a character, or a subword. No single tokenization is best for all languages and tasks. Converting everything to lowercase may simplify a topic classifier but can destroy useful distinctions such as proper names or product identifiers.

A bag-of-words representation counts tokens while discarding their order. For the vocabulary [robot, moves, stops], “robot moves” becomes [1, 1, 0], while “robot stops” becomes [1, 0, 1]. This representation cannot distinguish sentences with the same counts but different word order. N-grams preserve a limited amount of local order; embeddings map tokens or sequences to vectors whose learned structure may help with similarity, though geometric

closeness should not be treated as an infallible measure of meaning.

Probability and language models

A language model assigns probabilities to sequences. By the chain rule, $P(w_1, w_2) = P(w_1)P(w_2 \mid w_1)$, where w_1 and w_2 are tokens and the vertical bar means "given." If P(robot) = 0.1 and P(moves | robot) = 0.4, the joint probability of the two-token sequence under this simple model is 0.04. A bigram model approximates each next-token probability using the previous token. It cannot directly represent all long-range dependencies, and unseen bigrams require smoothing or another treatment.

Modern transformer models use attention to combine information across token positions. Their output probabilities are produced from learned parameters and context; generating plausible continuations is not equivalent to verifying a statement against the world. Systems that use such models for consequential tasks need independently checked evidence, restrictions on actions, and clearly defined human responsibility.

Classification, extraction, and evaluation

A sentiment classifier assigns a category to text, while named-entity recognition identifies spans such as names or places. For example, an entity system might mark "Lagos" in a transport request as a location. Correctness depends on both the span boundaries and the assigned class. A model can be right about the general topic and still misidentify the destination.

Evaluate on examples representative of actual use, including paraphrases, spelling errors, relevant languages, and ambiguous requests. Splitting near-duplicate messages across training and test sets can overstate generalization. For an action-taking interface, report not only classification metrics but also the rate of unsafe or misunderstood commands, and test the behavior when the system declines or requests clarification.

Language and robots

A language instruction must be grounded in the robot's environment: “the red cup” should be connected to an observed candidate object and a reachable location. Perception can be wrong, and a correct parse may still describe an unsafe action. A robust interaction confirms critical targets, checks permissions and physical constraints, and provides a way to stop. The system should tell users when it cannot reliably resolve a request rather than inventing certainty.

Review and investigation

1. Show two sentences with identical bag-of-words counts but different meanings. Explain what information is lost.

2. A language model assigns P(open) = 0.2 and P(door | open) = 0.3. Compute the probability of that two-token sequence under the given model.

3. Design a clarification question for an instruction involving two visually similar objects. Explain when the robot should refuse to act.

4. Compare evaluation of an entity extractor with evaluation of a complete robot instruction system.

6. Control Systems

From intention to feedback

Control engineering studies how to influence the behavior of dynamic systems. A controller may keep a motor at a desired speed or guide a robot arm to a position. In open-loop control, the command is issued without using a measurement of the resulting output to correct it. In closed-loop control, the measured output is compared with a reference and the difference informs subsequent action. Feedback can reduce some disturbances and model errors, but poor feedback design can also produce oscillation or instability.

Let r(t) be the desired output, y(t) the measured output, and e(t) = r(t) − y(t) the error at time t. If the desired speed is 10 revolutions per second and the measured speed is 8, the error is 2 revolutions per second. The sign convention matters: reversing the subtraction without adjusting the controller can turn negative feedback into positive feedback.

A simple dynamic model

A first-order plant can be written τ dy/dt + y = Ku, where y is output, u is input, K is the steady-state gain, and τ is a positive time constant. The units of K depend on the units of output and input; τ is measured in seconds when time is measured in seconds. For a constant input u and a stable plant at equilibrium, dy/dt = 0, so y = Ku. With K = 2 and u = 3, the steady-state output is 6 in the chosen output units.

For a step from zero initial output to a constant input, the ideal response is y(t) = Ku[1 − exp(−t/τ)]. At t = τ, the response has reached approximately 63.2% of its final value. This model is useful only when its assumptions approximate the real plant; actuator saturation, delay, changing load, and nonlinear friction may require a richer description.

Proportional and integral action

A proportional controller uses $u(t) = K_p e(t)$, where K_p is proportional gain. Increasing gain can produce a stronger response to error, but too much gain may cause oscillation or instability. For a unity-feedback first-order plant with steady-state gain K and no input saturation, the steady-state error for a constant reference r under proportional control is $e_{ss} = r/(1 + KK_p)$. If $K = 2$, $K_p = 3$ and $r = 7$, the error is 1 in output units. This formula depends on the stated structure and equilibrium assumptions; it is not a universal PID rule.

Integral action accumulates error over time. In continuous time, a standard PID expression is $u(t) = K_p e(t) + K_i \int e(t)dt + K_d\, de(t)/dt$. The integral gain is K_i and derivative gain is K_d. The integral term can remove steady-state error for suitable stable systems, while derivative action responds to the rate of change of error and can amplify measurement noise. The choice of units for each gain follows from the units of error, command, and time.

Actuators have limits. When a command is saturated, an unconstrained integral term may continue to grow, causing integral windup and a long recovery. Anti-windup methods limit or correct this accumulation. Practical implementations often filter derivative action and may differentiate the measurement rather than the reference to reduce derivative kick on setpoint changes.

Stability, sampling, and implementation

A stable response does not grow without bound after a bounded disturbance under the relevant stability definition and model assumptions. For a linear time-invariant continuous-time model, poles in the left half of the complex plane are a standard condition for asymptotic internal stability. A model is not the physical plant: an unmodeled delay, unexpected load, or broken sensor can invalidate a theoretical conclusion.

Digital controllers measure and update at discrete times. A simple discrete integral approximation is $I_k = I_{k-1} + K_i e_k \Delta t$, with sample interval Δt. Variable timing changes the accumulated contribution unless actual elapsed time is accounted for. Choose a sampling rate appropriate to system dynamics, check timing and filtering, and validate on a safe test setup. No fixed sampling rate is sufficient for all plants.

A cautious experiment

Start with a simulated first-order plant and a proportional controller. Record the reference, output, error, input, time step, and gain. Increase gain in small steps and compare rise time, overshoot, and steady-state error. Repeat with an actuator limit. Only after documenting model assumptions and conducting a physical risk assessment should a qualified team consider testing on hardware. Disconnecting software commands does not necessarily remove stored energy or bring moving equipment to a safe state.

Review and investigation

1. Distinguish a disturbance from measurement noise and give a robotics example of each.

2. For the first-order model with $K = 1.5$ and constant input 4, find the equilibrium output. State the assumptions.

3. Explain integral windup using a motor that cannot deliver the requested torque.

4. Draw a negative-feedback loop and label reference, error, controller, plant, measurement, and feedback direction.

Selected reading

The following established texts are starting points for further study; they are not a comprehensive bibliography for a finished volume. Consult the editions available to you and verify details before publication.

- Turing, Alan M. “Computing Machinery and Intelligence.” Mind 59, no. 236 (1950): 433–460. https://doi.org/10.1093/mind/LIX.236.433.

- Russell, Stuart, and Peter Norvig. Artificial Intelligence: A Modern Approach. 4th ed. Pearson, 2021. Chapters on agents, search, and learning.

- Siegwart, Roland, Illah R. Nourbakhsh, and Davide Scaramuzza. Introduction to Autonomous Mobile Robots. 2nd ed. MIT Press, 2011. Mobile platforms, sensing, and localization.

- Lynch, Kevin M., and Frank C. Park. Modern Robotics: Mechanics, Planning, and Control. Cambridge University Press, 2017. Kinematics and robot mechanics.

- Sutton, Richard S., and Andrew G. Barto. Reinforcement Learning: An Introduction. 2nd ed. MIT Press, 2018. Sequential decisions and reinforcement learning.

- Bishop, Christopher M. Pattern Recognition and Machine Learning. Springer, 2006. Statistical foundations and model evaluation.

- Szeliski, Richard. Computer Vision: Algorithms and Applications. 2nd ed. Springer, 2022. Image formation, features, and recognition.

- Jurafsky, Daniel, and James H. Martin. Speech and Language Processing. 2nd ed. Pearson, 2009. Language processing and statistical models.

- Åström, Karl J., and Richard M. Murray. Feedback Systems: An Introduction for Scientists and Engineers. Princeton University Press, 2008. Modeling, feedback, and stability.

Glossary

Agent: A system that selects actions using information from an environment.

Calibration: A procedure relating an instrument's indications to reference quantities, with attention to uncertainty.

Generalization: Performance on relevant examples not used to fit or select the model.

Grounding: Connecting a symbol or linguistic expression to entities or actions in a context.

Heuristic: An estimate used to guide a search; its formal properties depend on how it is defined.

Model: A simplified mathematical or computational representation used for a specified purpose.

Sensor: A device that produces a signal related to a physical quantity.

Stability: A property describing how a system responds to perturbations or inputs, under a specified definition.

Validation set: Data used to make modeling choices before final testing.